

Image Processing and Compression

Patrizia Stefani

March 12, 2022

Contents

1	Introduction	3
I	Vision	3
2	Human Visual Perception	3
2.1	Part I: Optical Components	4
2.2	Part II: Neural Components	5
2.3	Electromagnetic Spectrum	7
II	Image Processing	8
3	Image	8
3.1	Image storing	9
3.1.1	Colour representation	9
3.1.2	Image representation in MATLAB	10
3.1.3	Decreasing M and N	11
3.1.4	Decrease intensity's bits	13
3.2	Fourier Transform	18
3.2.1	Fourier Series	18
3.2.2	Towards the Fourier Transform	26
3.3	Inverse Fourier Transform	34
3.4	Discrete Fourier Transform	34
3.5	Sampling	36
3.5.1	Aliasing	39
3.6	Quantization	40
3.6.1	Lloyd-Max Quantizer	41
3.7	Filtering	42
3.7.1	Filtering in Spatial Domain	43
3.7.2	Filtering in Frequency Domain	46
4	Image Compression	49
4.1	Lossless compression	49
4.1.1	Huffman Coding	50
4.2	Lossy compression	59
4.2.1	SVD	59
4.2.2	Background on Matrices	59
4.2.3	Singular Value Decomposition	60
4.2.4	JPEG	68
4.3	Compression through Wavelet Transform	73
4.3.1	Wavelet Transform	73
4.4	Comparison and Complexity of Compression Algorithms	82

4.4.1	Huffman Coding	82
4.4.2	JPEG	82
4.4.3	Wavelets	82
4.4.4	SVD	83
4.5	Operations comparisons	83
4.6	Memory storage saving	84
5	Conclusion	84
A	A small Background in Trigonometry	85
B	Unitary Transforms	87
C	Stirling Approximation	88

1 Introduction

Nowadays the elaboration and compression of images is a very touchy area, as in the last decade multimedia contents have spread around at a consistent pace. Finding efficient and effective ways to store and transmit images without losing quality (or by finding a compromise between quality and transmission time), and also to be capable to reconstruct a distorted one has become a tough challenge. To deepen the topic of image processing and compression is my goal in this project, as I think it is a very fascinating field.

In particular, in the first part I wanted to have a brief overview of the functioning of the Human Visual Perception, which is a captivating subject, as it allows us to understand better ourselves, and also the choice of some techniques used in image processing.

After that I will provide some basic mathematical tools and definition, which will establish some foundations for the second part of the report about images, and after that I will focus on image sampling, quantization, processing, until reaching some compression techniques. The intent is to contextualize the theoretical concepts with practical examples, mainly developed in MATLAB, which is an environment that accompanied me during the whole project, and helped me to have a clearer view about the concepts treated.

Part I

Vision

2 Human Visual Perception

Before starting to dive deep in the image processing field, it would be interesting and useful to provide an overview of the Human Visual System (HVS), which, if understood well, can suggest and inspire some compression methods used in Computer Science to handle large image files.

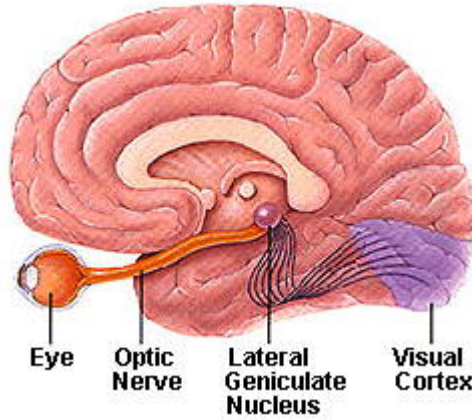


Fig. 1: Eye, Optic Nerve and Brain: the basic interacting components of the HVS.

Our HVS consists of two eyes, which are to be considered our "input sensors", the brain, which can be defined as an information processing unit, and the optic nerve, which allows the information to be carried out from the sensor to the processing unit (Figure 1).

2.1 Part I: Optical Components

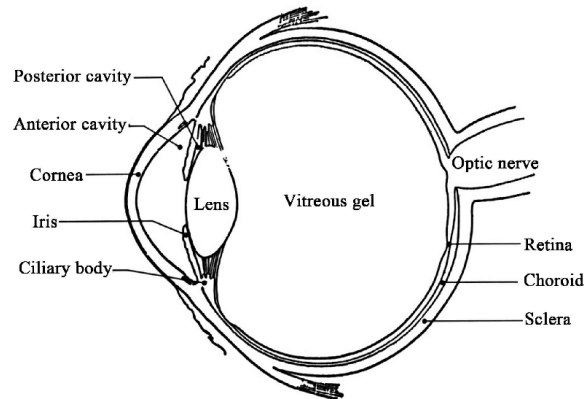


Fig. 2: Structure of the human eye

The eye lies in a *orbit*, which is a framework of bones and connective tissues, and is formed by some *transparent structures*, which are meant to achieve the maximum uninterrupted transmission of light and other types of structures. Between them we distinguish

- **the cornea**, which is a tough, transparent tissue retaining the most of the eye's focusing power. It is not equipped with blood vessels, as transparency is vital. An interesting curiosity is that the oxygen nearby dissolves in tears and diffuses throughout the cornea in order to maintain the health.
- **the lens**: they are concentric layers of fibrous cells, avascular and transparent. Their curvature can be modified by the contraction of ciliary muscles; the phenomenon of changing the lens curvature with the aim of focusing objects at different distances is called *accommodation*. Concerning their composition, it can be observed a high percentage of water (60-70%), 6% of fats, and a lot of proteins. The latter play a fundamental role, as they absorb considerably infrared and ultraviolet light: too much of these types of light could be deleterious for eyes. Furthermore, 8% of the visible light spectrum is absorbed by the lens. It has been discovered that a higher absorption occurs at shorter wavelengths, which is important in the study of image processing and compression, as it demonstrates that our eyes are less sensitive to higher wavelength details. This property is extensively exploited in computation techniques for image processing: for example, to compress large images low-frequency components could be recovered, while some high-frequencies are ignored.
- **the pupil** is defined as an aperture in the pigmented iris: its size can be varied by the smooth muscles of the iris, and as a consequence an alteration in the amount of light entering the eye is achieved. This so-called *pupil size adjustment* activity is based on the pupillary light reflex, which is controlled by the retinal illumination.
- **aqueous and vitreous humors**

These components together form the *optics* of the eye, and determine the quality of the image formed on the retina. This eye's layer structure is analogous to that of a camera in terms of its image-forming properties, so it is of main concern to understand it. All the components described can be clearly seen in Figure 2.

2.2 Part II: Neural Components

The innermost layer is the **retina**; when the eye is properly focused, the light from an object outside the eye is imaged on the retina.

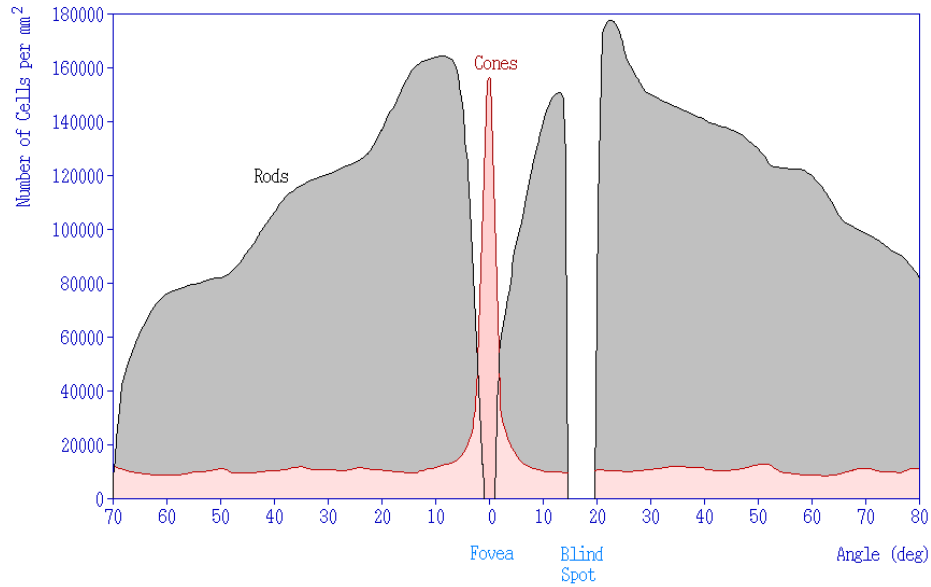


Fig. 3: Distribution of cones and rods in the eye's retina, based on the various degrees of view.

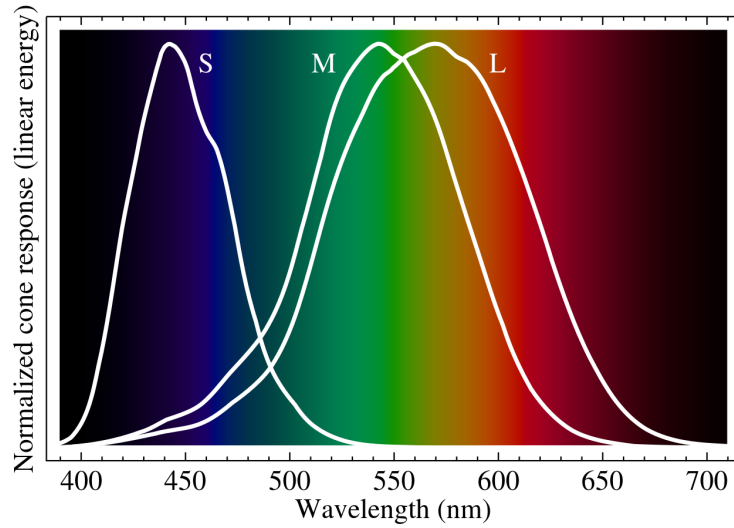


Fig. 4: S, M, L Cones receptors' sensitivities given different wavelengths: for example S cones respond more to the wavelengths located between 400 and 500 nm.

The retina is studded with two main types of discrete light receptors:

- **cones:** 6-7 millions, cover the central position of the retina, called fovea. They are characteristically highly sensitive to color and are more suitably used in an enlightened environment. For this reason they are connected with photopic or bright-light vision. Each cone cell has its own nerve end. We can distinguish between three types of cones, based on the wavelength they are capable to recognize: S, M, L, which correspond respectively to Short-wavelength, Medium-wavelength and Long-wavelength (they are represented in Figure 4).
- **rods:** they are between 75-100 millions, and they are spread over the whole retina, except from the blind spot, which can be seen in Figure 3 (between about 15 and 20 degrees), and the fovea, which is filled with cones. As opposed to the cones, several rods are connected to a single nerve, which means that rods cannot distinguish a high level of detail. Moreover, the rods recognize only colorless shapes and play a fundamental role in scotopic/dim-light vision.

2.3 Electromagnetic Spectrum

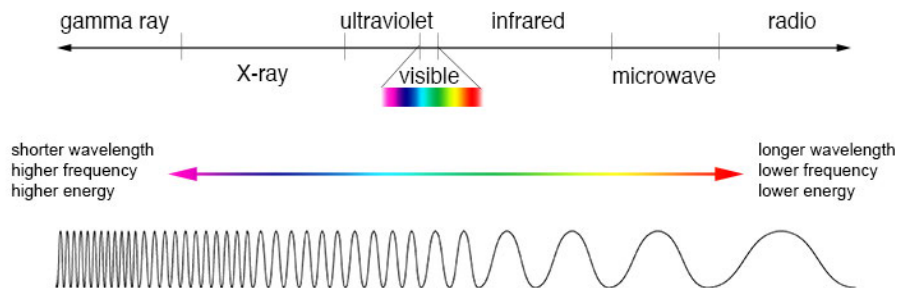


Fig. 5: Electromagnetic Spectrum: on the left side of the line we can see the gamma ray wavelength, which carry more energy, while on the extreme right side are located the radio wavelengths, which are lower frequency, and therefore they carry less energy.

It was previously discussed that the cones are highly sensitive to color, but what is color?

It was discovered that a beam of sunlight through a prism forms a continuous spectrum of colors, all contained between the boundary wavelengths *radio waves* (these waves are too long to be visible) and *gamma rays* (which are too short to be visible). Figure 5 describes the Electromagnetic Spectrum. Electromagnetic waves are propagating sinusoidal waves with wavelength λ (expressed in meters, m) and can be viewed as massless particles, which

contain a certain bundle of energy E (**photons**), measure in electron-volts. It is possible to view electromagnetic waves under different perspectives: wavelength (m), frequency (HZ) or energy (electron-volt). Given the speed of light c , the frequency ν and the Planck's constant h , we have the following relations:

$$\lambda = \frac{c}{\nu}, \text{ and}$$

$$E = h\nu.$$

From these equations it can be observed that higher-frequency waves (those who have shorter wavelength) carry more energy per photon. In fact *gamma rays* are the most dangerous type of light, as they are the highest-frequency components of the spectrum.

Colours are determined by the nature of light reflected by an object. For example, a green object reflects light with wavelengths in the range 500 and 570 nm, while absorbing most of the energy of other wavelengths.

Two types of light can be distinguished, in particular *achromatic/monochromatic* and *chromatic*. The first one is void in colour, and its tonality depends on the gray level, while the latter is seen by humans as coloured light and covers the color spectrum from about 0.43 to 0.79 μm of wavelengths. Chromatic light is formed by three components:

- **radiance**, measured in Watt, is the total amount of energy flowing from a light source;
- **luminance**, measured in Lumen, is the amount of energy perceived from a light source;
- **brightness** is a subjective parameter, which embodies the achromatic notion of intensity.

Part II

Image Processing

After having deepened concepts about HVS, it is now time to get started with the focus of this project, which is in images.

3 Image

What is an image? It can be regarded as a function $f(x, y)$ where $-\infty < x, y < \infty$. This function is the product of the two components *illuminance* and *reflectance*.

3.1 Image storing

We said that an image is a function $f(x, y)$, but in digital storage it has a particular structure, heavily used also in Linear Algebra. In particular, it is represented as a matrix, which, for each x and y , contains pixels' light intensities. The range of light intensities depends on how many bits are used to store the information: a typical value can be 8, meaning that one can store 2^8 different intensities, and this is enough for the human visual system sensitivity. This will be further discussed in the *Quantization* part (3.6).

If an image is stored in grey-scale, then it is sufficient to have just one matrix with the pixels' intensities, so the memory space used is $M \times N$, where M are the rows of the matrix, and N are the columns.

On the other hand, if we need to store a coloured image a common approach is to have three different matrices, each representing the intensities for the channels R (Red), G (Green), B (Blue). In this case the space needed is three times the one used by a grey-scale image (one matrix $M \times N$ for each channel): $M \times N \times 3$.

Another way to define an image is in terms of *vectors outer product*.

A vector outer product (considering two vectors of size N) between two vector is defined as following:

$$u_i v_j^T = \begin{pmatrix} u_{i1} \\ \dots \\ u_{iN} \end{pmatrix} (v_{j1} \quad \dots \quad v_{jN}) = \begin{pmatrix} u_{i1}v_{j1} & \dots & u_{i1}v_{jN} \\ \cdot & \dots & \cdot \\ \cdot & \dots & \cdot \\ \cdot & \dots & \cdot \\ u_{iN}v_{j1} & \dots & u_{iN}v_{jN} \end{pmatrix}$$

and this yields a matrix, which could be our image.

3.1.1 Colour representation

Previously it was mentioned that colours are stored in three distinct channels for each component red, green, and blue, but, often, to attain more sophisticated image qualities and a greater compression ratio, it is necessary to convert the RGB system into other coordinates which are closer and more suitable to the HVS, also because the manipulation of each RGB channel could lead to false colouring.

A popular colour representation which is particularly suitable is the YUV coordinate system [14], which divide the RGB components into *chroma* and *luma* components. Luma corresponds to the weighted sum of the R,G,B components, while the chroma is obtained by subtracting the luma from blue and red components. It is sometimes more convenient to utilize the YUV coordinate system than the RGB one, as the first provide more information

about the light intensity, and the human visual system is more sensitive to that than to the colours.

Hence, providing a practical representation, if we use the luma equation for the standard-definition television [14]:

$$Y = 0.299R + 0.587G + 0.114B. \quad (1)$$

Then, using Eq. (1), we can find the U and V components:

$$\begin{aligned} U &= B - Y = -0.299R - 0.587G + 0.886B, \\ V &= R - Y = 0.701R - 0.587G + 0.114B. \end{aligned} \quad (2)$$

At this stage we can summarize these results in matrix form

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.299 & -0.587 & 0.886 \\ 0.701 & -0.587 & -0.114 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}. \quad (3)$$

In the digital case, pixel values are contained in the range $[0, 255]$ with 8-bits, so there is a need to map levels to the appropriate digital value:

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 16 \\ 128 \\ 128 \end{bmatrix} + \begin{bmatrix} 65.481 & 128.553 & 24.996 \\ -37.797 & -74.203 & 112 \\ 112 & -93.786 & -18.214 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (4)$$

3.1.2 Image representation in MATLAB

Now a practical approach will be utilized, in order to show how images and colour are stored in the MATLAB environment.

Listing 1: Code to show the number of rows and columns and the number of channels for the matrix representation of an image

```

1 img = imread('img.jpg');
2 [rows, columns, channels] = size(img);
3 fprintf(Coloured image\nrows: %d columns: %d channels: %d\n\n,
   rows,columns,channels);
4
5 img = rgb2gray(img);
6 [rows, columns, channels] = size(img);
7 fprintf(Grey-scale image\nrows: %d columns: %d channels: %d\n\n,
   rows,columns,channels);

```

```

Coloured image
rows: 853  columns: 1280  channels: 3

Grey-scale image
rows: 853  columns: 1280  channels: 1

```

Fig. 6: It is shown the difference between the coloured image and the grey-scale image.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	76	78	78	72	79	74	75	76	79	76	77	73	81	79
2	78	78	75	69	77	76	80	85	91	71	69	81	88	84
3	76	76	75	74	81	76	77	82	81	82	75	79	88	87
4	74	75	75	78	83	78	74	78	77	77	82	87	76	82
5	78	78	73	76	80	79	78	85	80	81	73	82	87	84
6	80	80	73	75	78	79	76	83	86	76	76	84	82	86
7	77	81	75	78	79	80	73	75	79	85	83	76	82	85
8	75	80	75	78	79	83	76	78	84	77	79	79	83	79
9	76	78	80	76	78	83	79	81	82	79	86	80	85	95
10	78	80	81	76	78	82	79	80	81	75	81	82	86	90
11	78	80	80	76	78	82	79	81	81	79	79	81	82	83
12	77	78	79	75	78	82	79	82	81	86	83	80	79	83
13	78	78	80	77	79	82	80	83	83	88	85	83	84	88
14	81	81	82	79	81	83	80	83	88	84	84	85	87	86
15	81	81	82	81	83	84	80	83	87	81	85	84	85	87
16	78	78	81	80	83	84	80	84	81	80	88	81	82	93
17	84	80	77	84	86	84	86	81	77	87	90	83	84	82
18	75	89	88	78	72	79	89	88	80	84	82	85	86	88
19	81	89	85	82	81	77	82	89	81	86	81	86	84	91
20	88	85	77	85	92	81	79	88	80	94	91	86	81	89
21	81	89	82	79	85	85	87	90	82	93	95	86	86	88
22	80	91	87	83	87	90	90	86	85	85	86	85	91	86
23	86	85	82	90	94	90	88	84	86	85	83	87	91	88
24	83	78	76	85	88	87	91	90	88	95	91	93	87	92
25	88	82	78	92	83	80	84	83	84	85	95	87	85	94

Fig. 7: Here the matrix structure of the image

An important aspect to consider is that increasing M and N will improve the resolution of the image, while reducing them will affect it in a negative way: in fact, if M and N are too small, we will encounter the *checkboard effect* by seeing a pixelated image.

Decreasing the number of bits used to store the intensity levels will also have an effect on the image, but at first it will not be visible, just with a radical reduction of them we will have a *false contouring* effect. For example, if we use only 1 bit, we will just see black or white pixels in our image, and some contourings will not be the actual ones. An interesting aspect is that an image containing a lot of details will not be affected so much by the decrease of intensity's bits.

Now let's check out some examples.

3.1.3 Decreasing M and N

Listing 2: Image quality degradation as a consequence of reducing the number of rows and columns, observed by plotting the original and the scaled pictures at the same window size.

```
1 %read image and show it
2 img = rgb2gray(imread('detailedImg.jpg'));
3 figure;
4 imshow(img);
5
6 %decrease number of rows and columns
7 [r,c] = size(img);
8 imgDegraded = img(1:4:r, 1:4:c);
9 imgDegraded = imresize(imgDegraded, [r,c]);
10
11 %show degraded image
12 figure;
13 imshow(imgDegraded);
```



Fig. 8: The original image



Fig. 9: Lower resolution image

3.1.4 Decrease intensity's bits

I use the same cat image to see how the decrease of the intensity levels is affecting the image, by considering that the original image's intensities are stored with 8 bits (256 levels).

Listing 3: Image quality degradation when only 5 intensity levels are used in the image.

```
1 %read image and show it
2 img = rgb2gray(imread('detailedImg.jpg'));
3 figure;
4 imshow(img);
5
6 %quantize at lower levels
7 %number of levels: 5
8 %level offset: 256/5
9 imgQuantized = zeros(r,c);
10 lvlOffset = 256/5;
11 for i = 1:r
12     for j = 1:c
13         if(img(i,j) <= lvlOffset)
14             imgQuantized(i,j) = floor(lvlOffset);
15         end
16         if(img(i,j) > lvlOffset && img(i,j) <= lvlOffset*2)
```

```

17         imgQuantized(i,j) = floor(lvlOffset*2);
18     end
19     if(img(i,j) > lvlOffset*2 && img(i,j) <= lvlOffset*3)
20         imgQuantized(i,j) = floor(lvlOffset*3);
21     end
22     if(img(i,j) > lvlOffset*3 && img(i,j) <= lvlOffset*4)
23         imgQuantized(i,j) = floor(lvlOffset*4);
24     end
25     if(img(i,j) > lvlOffset*4 && img(i,j) <= lvlOffset*5)
26         imgQuantized(i,j) = floor(lvlOffset*5);
27     end
28 end
29 end
30 figure;
31 imshow(imgQuantized,[]);
32
33 %quantize at lower levels
34 %number of levels: 2
35 %level offset: 256/2
36 lvlOffset = 256/2;
37 for i = 1:r
38     for j = 1:c
39         if(img(i,j) <= lvlOffset)
40             imgQuantized(i,j) = floor(lvlOffset);
41         end
42         if(img(i,j) > lvlOffset)
43             imgQuantized(i,j) = floor(lvlOffset*2);
44         end
45     end
46 end
47 figure;
48 imshow(imgQuantized,[]);

```



Fig. 10: The original image.



Fig. 11: The same image, using only five possible levels per pixel.



Fig. 12: image with two levels: the difference is very noticeable.

However, it should be noted that when the number of bits used is significantly reduced, the overall aspect of an image is preserved and its details are still distinguishable.



Fig. 13: Original detailed image



Fig. 14: Image represented through five levels

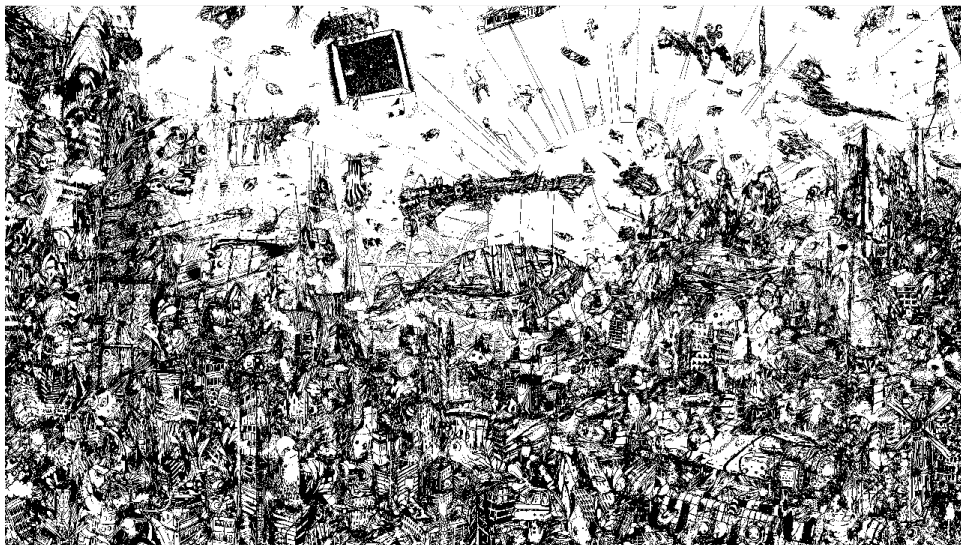


Fig. 15: Image represented through two levels, as you can observe, details are still visible

To understand image sampling, an important mathematical tool, the Fourier Transform needs to be introduced, namely Fourier analysis, and this will be done in the next section.

3.2 Fourier Transform

One of the most important mathematical concepts in image processing is the Fourier Transform, as it allows to represent the image function in the frequency domain rather than in the spatial domain. This property can be useful to design filters that reduce noise.

The concrete meaning of an image pictured in the Frequency Domain is the rate of change of the intensity per pixel. Therefore, the longer it takes for a color to change, the lower the frequency, while high-frequency components represent an abrupt change in pixels intensities.

The Fourier Transform can be seen as a limit case of the Fourier Series, so it is important to have an overall view of the Fourier Series.

3.2.1 Fourier Series

The Fourier Series allows a periodic function to be represented as through a sum of sines and cosines.

To derive the Fourier Series it is possible to break it down in different pieces: the case of sines sum, the case of a cosines sum, and then a combination of them.

Definition (Periodic Function). *A function is defined as periodic if there exist a nonzero value P for which $f(x + P) = f(x)$.*

Lemma 1 (Sines sum). *A periodic function can be expressed as a sum of sines as follows:*

$$f(x) = \sum_{n=1}^{\infty} A_n \sin\left(\frac{n\pi x}{l}\right), \quad 0 < x < l,$$

with unique coefficients

$$Am = \frac{2}{l} \int_0^l f(x) \sin\left(\frac{m\pi x}{l}\right) dx, \quad \text{for every } m \geq 0. \quad (5)$$

Proof. To ensure that the equation holds and that the coefficients are unique for every proper m , we can fix an integer $m \geq 0$, multiply function $f(x)$ by $\sin\left(\frac{m\pi x}{l}\right)$, and integrate term by term

$$\begin{aligned} \int_0^l f(x) \sin\left(\frac{m\pi x}{l}\right) dx &= \int_0^l \sum_{n=1}^{\infty} A_n \sin\left(\frac{n\pi x}{l}\right) \sin\left(\frac{m\pi x}{l}\right) dx = \\ &= \sum_{n=1}^{\infty} A_n \int_0^l \sin\left(\frac{n\pi x}{l}\right) \sin\left(\frac{m\pi x}{l}\right) dx. \end{aligned} \quad (6)$$

But, as seen in Section A, Lemma 4

$$\sin(a) \sin(b) = \frac{1}{2} \cos(a - b) - \frac{1}{2} \cos(a + b).$$

On expanding Equation Eq. (6) as

$$\begin{aligned}\int_0^l f(x) \sin\left(\frac{m\pi x}{l}\right) dx &= \int_0^l \sum_{n=1}^{\infty} A_n \sin\left(\frac{n\pi x}{l}\right) \sin\left(\frac{m\pi x}{l}\right) dx = \\ &= \sum_{n=1}^{\infty} A_n \int_0^l \sin\left(\frac{n\pi x}{l}\right) \sin\left(\frac{m\pi x}{l}\right) dx;\end{aligned}$$

and replacing $\sin\left(\frac{n\pi x}{l}\right) \sin\left(\frac{m\pi x}{l}\right)$ with $\frac{1}{2} \cos\left(\frac{n\pi x}{l} - \frac{m\pi x}{l}\right) - \cos\left(\frac{n\pi x}{l} + \frac{m\pi x}{l}\right)$ we obtain

$$\begin{aligned}&\sum_{n=1}^{\infty} A_n \int_0^l \frac{1}{2} \cos\left(\frac{n\pi x - m\pi x}{l}\right) - \frac{1}{2} \cos\left(\frac{n\pi x + m\pi x}{l}\right) dx = \\ &= \sum_{n=1}^{\infty} A_n \left(\frac{1}{2} \int_0^l \cos\left(\frac{n\pi x - m\pi x}{l}\right) dx - \frac{1}{2} \int_0^l \cos\left(\frac{n\pi x + m\pi x}{l}\right) dx \right) = \\ &= \sum_{n=1}^{\infty} A_n \left(\frac{1}{2} \int_0^l \cos\left(\frac{\pi x(n - m)}{l}\right) dx - \frac{1}{2} \int_0^l \cos\left(\frac{\pi x(n + m)}{l}\right) dx \right).\end{aligned}$$

By integration by substitution, we introduce the following variables

$$\begin{aligned}u &= \frac{\pi x(n - m)}{l}, \\ \frac{du \cdot l}{\pi(n - m)} &= dx; \\ v &= \frac{\pi x(n + m)}{l}, \\ \frac{dv \cdot l}{\pi(n + m)} &= dx,\end{aligned}$$

and then solve the integral

$$\begin{aligned}
& \sum_{n=1}^{\infty} A_n \left(\frac{1}{2} \int_0^l \cos(u) \frac{du \cdot l}{\pi(n-m)} - \frac{1}{2} \int_0^l \cos(v) \frac{dv \cdot l}{\pi(n+m)} \right) = \\
& = \sum_{n=1}^{\infty} A_n \left(\frac{l}{2\pi(n-m)} \int_0^l \cos(u) du - \frac{l}{2\pi(n+m)} \int_0^l \cos(v) dv \right) = \\
& = \sum_{n=1}^{\infty} A_n \left(\frac{l}{2\pi(n-m)} \sin\left(\frac{\pi \cdot l(n-m)}{l}\right) - \frac{l}{2\pi(n+m)} \sin\left(\frac{\pi \cdot l(n+m)}{l}\right) \right) = \\
& = \sum_{n=1}^{\infty} A_n \left(\frac{l}{2\pi(n-m)} \sin(\pi \cdot (n-m)) - \frac{l}{2\pi(n+m)} \sin(\pi \cdot (n+m)) \right) = 0.
\end{aligned}$$

If $m \neq n$ then this equals 0, as the sine is 0 at multiples of π .
On the other hand, if $m = n$, Equation 6 reduces to $\sin^2\left(\frac{m\pi x}{l}\right)$ and the the following integral should be computed:

$$A_m \int_0^l \sin^2\left(\frac{m\pi x}{l}\right) dx. \quad (7)$$

$\int \sin^2(ax)$ is equal to $\frac{x}{2} - \frac{1}{4a} \sin(2ax) + c$, hence, integral 7 is

$$\begin{aligned}
\int_0^l \sin^2\left(\frac{m\pi x}{l}\right) dx &= \frac{l}{2} - \frac{1}{4m\pi} \sin(2m\pi) \rightarrow \\
\int_0^l f(x) \sin\left(\frac{m\pi x}{l}\right) dx &= A_m \left(\frac{l}{2} - \frac{1}{4m\pi} \sin(2m\pi) \right) = A_m \cdot \frac{l}{2},
\end{aligned}$$

since the sine is 0 at angles multiples of π .

Therefore, we can write

$$\int_0^l f(x) \sin\left(\frac{m\pi x}{l}\right) dx = A_m \cdot \frac{l}{2},$$

and finally

$$Am = \frac{2}{l} \cdot \int_0^l f(x) \sin\left(\frac{m\pi x}{l}\right) dx.$$

□

After having described the sines form of a periodic function, the cosines sum will be derived.

Lemma 2 (Cosines sum). *We can write the function as a sum of cosines*

$$f(x) = \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} A_n \cos\left(\frac{n\pi x}{l}\right), \quad 0 < x < l,$$

with unique coefficients

$$A_m = \frac{2}{l} \int_0^l f(x) \cos\left(\frac{m\pi x}{l}\right) dx, \quad \text{for every } m \geq 0. \quad (8)$$

Proof. To prove this, it is enough to apply the same procedure described in Lemma 1, it is necessary to consider $f(x)$, multiply it by $\cos(\frac{m\pi x}{l})$, where m is an integer greater or equal to 0, and integrate term by term:

$$\int_0^l f(x) \cos\left(\frac{m\pi x}{l}\right) dx = \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} A_n \int_0^l \cos\left(\frac{n\pi x}{l}\right) \cos\left(\frac{m\pi x}{l}\right) dx.$$

We can develop the product of $\cos(a)\cos(b)$ in this way:

$$\cos(a)\cos(b) = \frac{1}{2} \cos(a-b) + \frac{1}{2} \cos(a+b).$$

Hence,

$$\begin{aligned} \int_0^l f(x) \cos\left(\frac{m\pi x}{l}\right) dx &= \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} A_n \left(\frac{1}{2} \int_0^l \cos\left(\frac{n\pi x - m\pi x}{l}\right) dx + \right. \\ &\quad \left. \frac{1}{2} \int_0^l \cos\left(\frac{n\pi x + m\pi x}{l}\right) dx \right) = \\ &= \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} A_n \left(\frac{1}{2} \int_0^l \cos\left(\frac{\pi x(n-m)}{l}\right) dx + \frac{1}{2} \int_0^l \cos\left(\frac{\pi x(n+m)}{l}\right) dx \right) = \\ &= \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} A_n \left(\left[\frac{l}{2\pi(n-m)} \sin\left(\frac{\pi x \cdot (n-m)}{l}\right) \right]_0^l + \left[\frac{l}{2\pi(n+m)} \sin\left(\frac{\pi x \cdot (n+m)}{l}\right) \right]_0^l \right) = \\ &= \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} A_n \left(\frac{l}{2\pi(n-m)} \sin(\pi \cdot (n-m)) + \frac{l}{2\pi(n+m)} \sin(\pi \cdot (n+m)) \right). \end{aligned}$$

It can be observed that when $n \neq m$ this quantity is equal to 0, as the sine of multiples of π is 0.

On the other hand, if $n = m$ and $m \neq 0$ the integral from 0 to l of $f(x) \cos(\frac{m\pi x}{l})$ with $n = m$ with respect to x should be computed.

As $\int \cos^2(ax) = \frac{x}{2} + \frac{1}{4a} \sin(2ax) + c$ (as proved in Section A, Lemma 5), the

integral can be expanded

$$A_m \int_0^l \cos^2 \left(\frac{m\pi x}{l} \right) dx = A_m \left(\frac{l}{2} + \frac{l}{4m\pi} \sin \left(2m\pi \right) \right).$$

But $\sin(2m\pi)$ is always 0, hence

$$A_m \int_0^l \cos^2 \left(\frac{m\pi x}{l} \right) dx = A_m \left(\frac{l}{2} \right)$$

$$A_m = \frac{l}{2} \int_0^l f(x) \cos \left(\frac{m\pi x}{l} \right) dx$$

If $m = 0$, then

$$A_0 = \frac{l}{2} \int_0^l f(x) \cdot 1 dx = \frac{l}{2} \int_0^l f(x) dx.$$

□

Lemma 3 (Full Fourier Series). *Combining the sines and cosines sums, with get this other definition: a function or signal $f(x)$ can be expressed as a sum of sines and cosines in the interval $-l < 0 < l$ as*

$$f(x) = \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} \left(A_n \cos \left(\frac{n\pi x}{l} \right) + B_n \sin \left(\frac{n\pi x}{l} \right) \right), \quad (9)$$

with unique coefficients

$$\begin{aligned} A_n &= \frac{1}{l} \int_{-l}^l f(x) \cos \left(\frac{n\pi x}{l} \right) dx \quad n \geq 0, \\ B_n &= \frac{1}{l} \int_{-l}^l f(x) \sin \left(\frac{n\pi x}{l} \right) dx \quad n \geq 0. \end{aligned} \quad (10)$$

Proof. Why does it hold? Let's take $f(x)$, multiply it by $\cos \left(\frac{m\pi x}{l} \right)$, with $m \geq 0$ and integrate from $-l$ to l

$$\begin{aligned} \int_{-l}^l f(x) \cos \left(\frac{m\pi x}{l} \right) dx &= \frac{1}{2} \cdot A_0 + \int_{-l}^l \sum_{n=1}^{\infty} \left(A_n \cos \left(\frac{n\pi x}{l} \right) \cos \left(\frac{m\pi x}{l} \right) + B_n \sin \left(\frac{n\pi x}{l} \right) \cos \left(\frac{m\pi x}{l} \right) \right) dx \\ &= \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} A_n \int_{-l}^l \cos \left(\frac{n\pi x}{l} \right) \cos \left(\frac{m\pi x}{l} \right) dx + \sum_{n=1}^{\infty} B_n \int_{-l}^l \sin \left(\frac{n\pi x}{l} \right) \cos \left(\frac{m\pi x}{l} \right) dx. \end{aligned} \quad (11)$$

The second sum term must be elaborated, considering the following equality

$$\cos(\alpha) \sin(\beta) = \frac{\sin(\alpha + \beta) - \sin(\alpha - \beta)}{2}; \quad (12)$$

and the fact that cosine is an even function,

$$\cos(x) = \cos(-x);$$

we can plug this in the integral

$$\begin{aligned} & \sum_{n=1}^{\infty} B_n \left(\frac{1}{2} \int_{-l}^l \sin \left(\frac{n\pi x + m\pi x}{l} \right) dx - \frac{1}{2} \int_{-l}^l \sin \left(\frac{n\pi x - m\pi x}{l} \right) dx \right) = \\ &= \sum_{n=1}^{\infty} \frac{B_n}{2} \left(\left[-\frac{l \cdot \cos \left(\frac{\pi x(m+n)}{l} \right)}{\pi(m+n)} \right]_{-l}^l - \left[-\frac{l \cdot \cos \left(\frac{\pi x(m-n)}{l} \right)}{\pi(m-n)} \right]_{-l}^l \right) = \\ &= \sum_{n=1}^{\infty} \frac{B_n}{2} \left(\left[-\frac{l \cdot \cos(\pi \cdot (m+n))}{\pi(m+n)} + \frac{l \cdot \cos(-\pi \cdot (m+n))}{\pi(m+n)} \right] - \right. \\ & \quad \left. \left[-\frac{l \cdot \cos(\pi \cdot (m-n))}{\pi(m-n)} + \frac{l \cdot \cos(-\pi \cdot (m-n))}{\pi(m-n)} \right] \right) = \\ &= \sum_{n=1}^{\infty} \frac{B_n}{2} \left(-\frac{l \cdot \cos(\pi \cdot (m+n))}{\pi(m+n)} + \frac{l \cdot \cos(-\pi \cdot (m+n))}{\pi(m+n)} + \right. \\ & \quad \left. \frac{l \cdot \cos(\pi \cdot (m-n))}{\pi(m-n)} - \frac{l \cdot \cos(-\pi \cdot (m-n))}{\pi(m-n)} \right). \quad (13) \end{aligned}$$

Cosine is even

$$\frac{l \cdot \cos(-\pi \cdot (m+n))}{\pi(m+n)} = \frac{l \cdot \cos(\pi \cdot (m+n))}{\pi(m+n)},$$

consequently

$$\frac{l \cdot \cos(\pi \cdot (m-n))}{\pi(m-n)} = \frac{l \cdot \cos(-\pi \cdot (m-n))}{\pi(m-n)},$$

and all the terms will vanish.

On the other hand the first term of the sum can be expanded as follows

$$\begin{aligned} & \sum_{n=1}^{\infty} \frac{A_n}{2} \left(\left[\frac{l \cdot \sin \left(\frac{\pi x(n-m)}{l} \right)}{\pi(n-m)} \right]_{-l}^l + \left[\frac{l \cdot \sin \left(\frac{\pi x(n+m)}{l} \right)}{\pi(n+m)} \right]_{-l}^l \right) = \\ &= \sum_{n=1}^{\infty} \frac{A_n}{2} \left(\left[\frac{l \cdot \sin(\pi(n-m))}{\pi(n-m)} - \frac{l \cdot \sin(-\pi(n-m))}{\pi(n-m)} \right] + \left[\frac{l \cdot \sin(\pi(n+m))}{\pi(n+m)} - \frac{l \cdot \sin(-\pi(n+m))}{\pi(n+m)} \right] \right). \end{aligned} \quad (14)$$

Sine is an odd function: $\sin(x) = -\sin(-x)$, hence, developing the equation further will flow in this way:

$$\sum_{n=1}^{\infty} \frac{A_n}{2} \left(\frac{2 \cdot l \cdot \sin(\pi(n-m))}{\pi(n-m)} + \frac{2 \cdot l \cdot \sin(\pi(n+m))}{\pi(n+m)} \right).$$

If $n \neq m$, the sines of multiples of π are evaluated, therefore the terms are fading away.

If $n = m$ we must evaluate the integral of $f(x) \cos(\frac{m \cdot \pi x}{l})$ with respect to x , with the limits of integration $-l$ and l :

$$\begin{aligned} A_m \int_{-l}^l \cos^2 \left(\frac{m \pi x}{l} \right) dx &= A_m \left(\frac{l}{2} - \left(-\frac{l}{2} \right) \right) \rightarrow \\ A_m \int_{-l}^l \cos^2 \left(\frac{m \pi x}{l} \right) dx &= A_m \left(\frac{2 \cdot l}{2} \right) \rightarrow \\ A_m &= \frac{1}{2} \int_{-l}^l f(x) \cos \left(\frac{m \pi x}{l} \right) dx. \end{aligned} \tag{15}$$

The same mechanism could be used to prove that $B_n \sin \left(\frac{n \pi x}{l} \right)$ has unique coefficients $B_n = \frac{1}{l} \int_{-l}^l f(x) \sin \left(\frac{n \pi x}{l} \right) dx \quad n \geq 0$.

□

The Full Fourier Series can be formulated in the equivalent complex form, which could be advantageous for calculations. The following Theorem will show how this is achieved.

Theorem 1 (Complex Full Fourier Series). *Sines and cosines terms can be rewritten in their complex form*

$$f(x) = \sum_{n=-\infty}^{\infty} c_n e^{\frac{in \pi x}{l}}, \tag{16}$$

starting from the Euler's Formula

$$e^{i\theta} = \cos(\theta) + i \sin(\theta).$$

Proof. To find the single terms $\cos(\theta)$ and $\sin(\theta)$ we can proceed in this way:

Considering that

$$e^{-i\theta} = \cos(-\theta) - i \sin(\theta);$$

summing $e^{i\theta}$ and $e^{-i\theta}$ we get

$$e^{i\theta} + e^{-i\theta} = \cos(\theta) + i \sin(\theta) + \cos(-\theta) - i \sin(\theta).$$

As cosine is an even function

$$\begin{aligned} e^{i\theta} + e^{-i\theta} &= 2 \cos(\theta), \\ \cos(\theta) &= \frac{e^{i\theta} + e^{-i\theta}}{2}. \end{aligned} \tag{17}$$

Subtracting the two expressions $e^{i\theta}$ and $e^{-i\theta}$, we get

$$\begin{aligned} e^{i\theta} - e^{-i\theta} &= \cos(\theta) + i \sin(\theta) - \cos(-\theta) + i \sin(\theta) = \\ &= \cos(\theta) + i \sin(\theta) - \cos(\theta) + i \sin(\theta) = \\ &= 2i \sin(\theta) \\ \sin(\theta) &= \frac{e^{i\theta} - e^{-i\theta}}{2i}. \end{aligned} \tag{18}$$

Therefore, the Full Fourier Series given in Lemma (3) could be rewritten in the form

$$f(x) = \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} \left(A_n \frac{e^{i\theta} + e^{-i\theta}}{2} + B_n \frac{e^{i\theta} - e^{-i\theta}}{2i} \right).$$

Factoring the quantities

$$e^{i\theta}$$

and

$$e^{-i\theta}$$

gives

$$\begin{aligned} f(x) &= \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} \left(\left(\frac{A_n}{2} + \frac{B_n}{2i} \right) e^{i\theta} + \left(\frac{A_n}{2} - \frac{B_n}{2i} \right) e^{-i\theta} \right) = \\ &= \frac{1}{2} \cdot c_0 + \sum_{n=1}^{\infty} \left(c_n e^{i\theta} + d_n e^{-i\theta} \right), \end{aligned}$$

with

$$c_0 = A_0, c_n = \frac{A_n}{2} + \frac{B_n}{2i}, \quad n \geq 1,$$

and

$$d_n = \frac{A_n}{2} - \frac{B_n}{2i}, \quad n \geq 1,$$

or, shortly

$$f(x) = \sum_{n=-\infty}^{\infty} c_n e^{\frac{in\pi x}{l}},$$

with unique coefficients

$$c_n = \frac{1}{2l} \int_{-l}^l f(x) e^{\frac{-in\pi x}{l}} dx.$$

□

3.2.2 Towards the Fourier Transform

The Fourier Transform can be applied also to non periodic functions, by treating them as periodic functions where the period extends towards infinity. More precisely, a non periodic function can be periodized in two different ways, either evenly or oddly. The following short MATLAB script, plots a constant function of constant value 1 in the range $[-1.5, 1.5]$ (the interval chosen could be regarded as a shift of $[0, l]$, where l is 3 and by shifting the function by 1.5 to the left we get the desired range $[-1.5, 1.5]$), and then periodize it evenly and oddly.

Listing 4: Script which represents the signal even and then odd periodized, given that the original is defined in the intervals $-1.5 < x < 1.5$ and it is 1

```
1 syms x;  
2 y = piecewise(x > -1.5 & x < 1.5, 1);  
3 figure;  
4 fplot(y, [-3, 3]); ylim([-3, 3]);  
5  
6 y = piecewise(mod(x, 2) < 1 & x > -3 & x < 3, 1, mod(x, 2) > 1 & x  
7 > -3 & x < 3, -1);  
8 figure;  
9 fplot(y, [-3, 3]); ylim([-3, 3]);  
10  
11 y = piecewise(x > -3 & x < 3, 1);
```

```

11 figure;
12 fplot(y,[-3,3]);ylim([-3,3]);

```

Firstly, the even periodization will be described.

Definition (Even periodization). *As already mentioned, an even function is a function which satisfies the following condition:*

$$f(x) = f(-x),$$

an example is the cosine function.

The function $f(x)$ defined in an interval $0 < x < l$ has an even periodic extension $F^e(x)$ defined in an interval $[-L, L]$ if the following conditions are complied with:

Conditions.

- $F^e(x) = f(x), \quad 0 < x < l;$
- $F^e(x)$ is even, which implies that $F^e(x)$ is defined also in $-l < x < 0;$
- $F^e(x)$ has period $2l.$

Figure 16 contains the graph of the original signal periodized in an even manner.

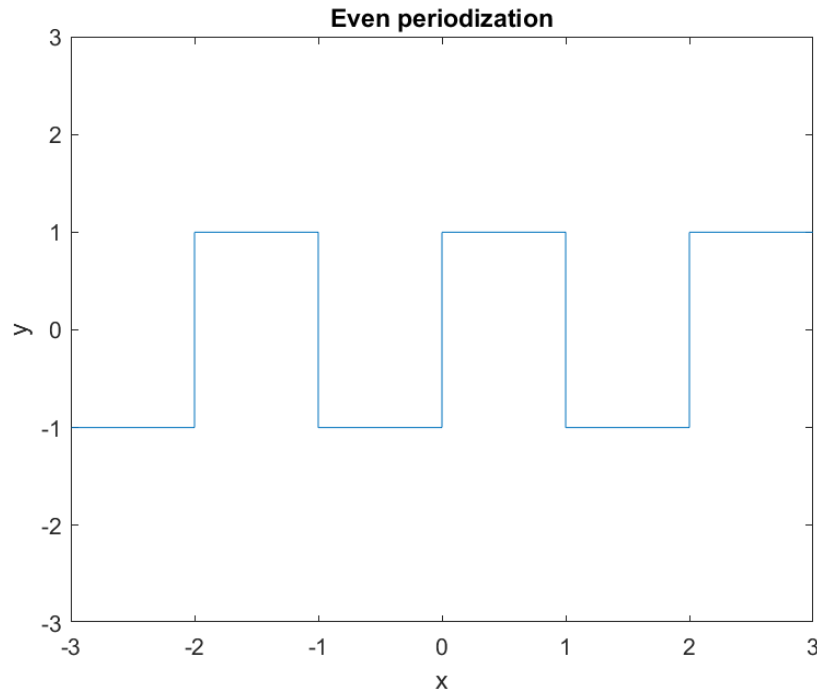


Fig. 16: Function $f(x)$ defined in the interval $[-1.5, 1.5]$ even periodized in the interval $[-3, 3]$

Secondly, a description of the odd periodization.

Definition (Odd periodization). *An odd function, instead, is a function which satisfies the following condition:*

$$f(-x) = -f(x).$$

An example is the sine function.

The function $f(x)$ defined in an interval $0 < x < l$ has an odd periodic extension $F^o(x)$ defined in an interval $[-L, L]$ if the following conditions are complied with:

Conditions.

- $F^o(x) = f(x), \quad 0 < x < l;$
- $F^o(x)$ is odd;
- $F^o(x)$ has period $2l$.

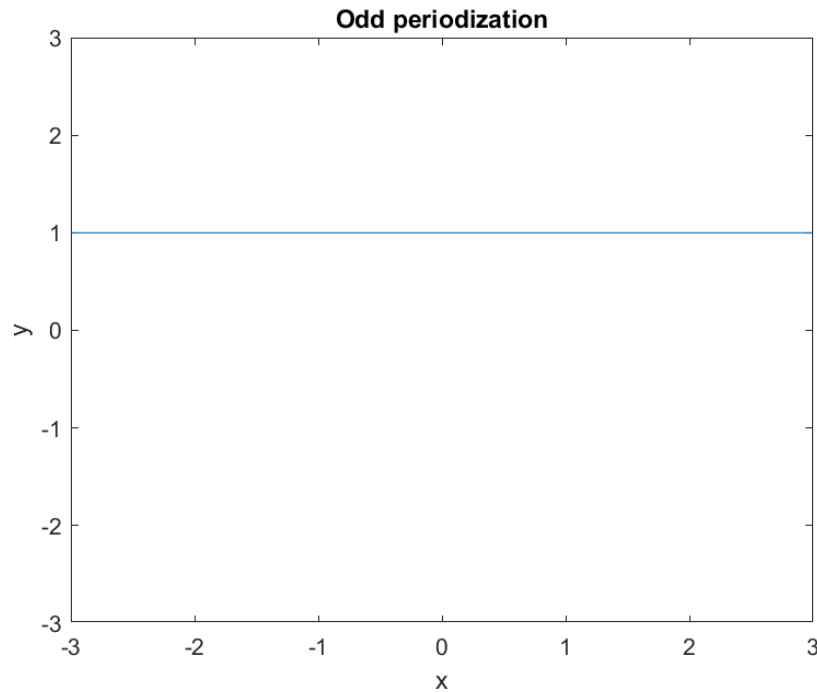


Fig. 17: Function $f(x)$ defined in the interval $[-1.5, 1.5]$ oddly periodized in the interval $[-3, 3]$

As mentioned in the properties for odd and even extension of $f(x)$, it is known that $f(x) = F^e(x) = F^o(x)$, and the even extension can be expanded through the sines sum, while the odd extension can be represented through the cosines sum.

Hence:

$$f(x) = F^e(x) = \frac{1}{2} \cdot A_0 + \sum_{n=1}^{\infty} A_n \cos\left(\frac{n\pi x}{l}\right) = F^o(x) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{l}\right),$$

with coefficients:

$$\begin{aligned} A_m &= \frac{2}{l} \int_0^l F^e(x) \cos\left(\frac{m\pi x}{l}\right) dx = \frac{2}{l} \int_0^l f(x) \cos\left(\frac{m\pi x}{l}\right) dx, & \text{for every } m \geq 0; \\ B_m &= \frac{2}{l} \int_0^l F^o(x) \sin\left(\frac{m\pi x}{l}\right) dx = \frac{2}{l} \int_0^l f(x) \sin\left(\frac{m\pi x}{l}\right) dx, & \text{for every } m \geq 0. \end{aligned} \quad (19)$$

We then represent the periodic extension of the Fourier Series in the interval $[-l, l]$ (extension of $f(x)$) in this way:

Definition.

$$f_{\text{periodic}}(x) = f(x - 2lm), \quad -l + 2lm < x < l + 2lm, \quad m \in \mathbb{N}. \quad (20)$$

In [6] we can find an example, which combines the previous described periodizations to reach the Fourier transform for a non periodic signal (Figure 18): in fact, through the periodization we can discover some interesting facts about the correlation of period and Fourier coefficients.

$$\Pi(x) \begin{cases} 1 & |t| < \frac{1}{2} \\ 0 & |t| > \frac{1}{2} \end{cases} \quad (21)$$

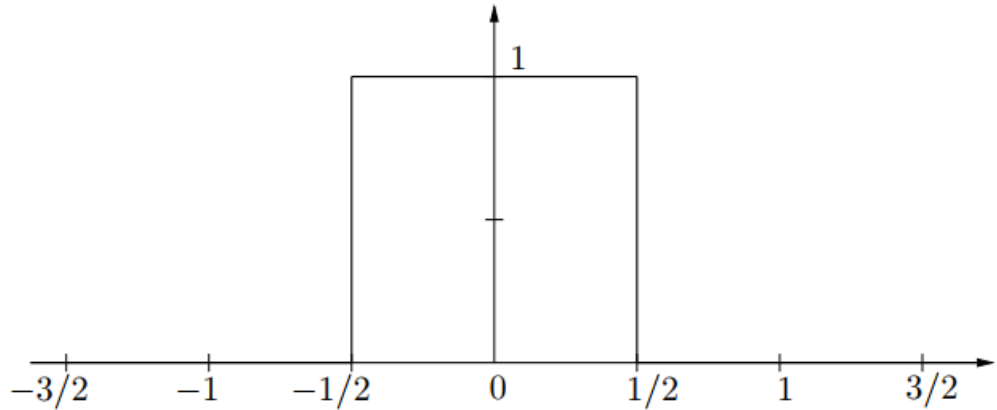


Fig. 18: A pulse π . No period is observable

Although the function is not periodic, the textbook suggests to periodize it and try to focus on the graphs representing the coefficients of the Fourier series, given the same function periodized at different lengths. Firstly, the period for the pulse is 2, as in Figure 19.

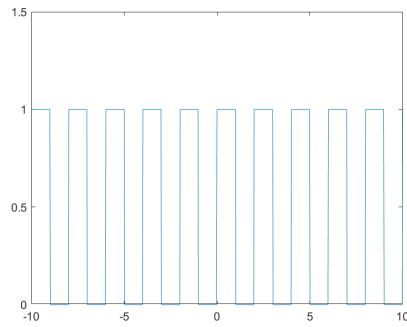


Fig. 19: Function $\Pi(x)$ periodized with period 2

And the interesting result observable in the Fourier coefficients is the following:

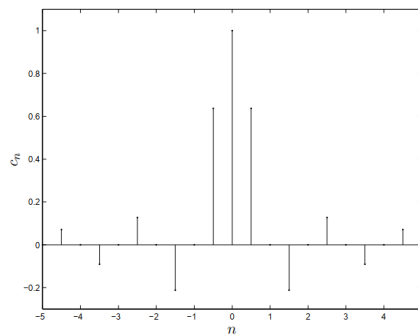


Fig. 20: Fourier coefficients of the function $\Pi(x)$'s with period 2

Changing the period to 4 the function looks like in Figure 21.

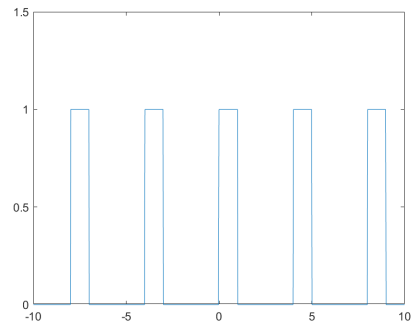


Fig. 21: Function $\Pi(x)$ periodized with period 4

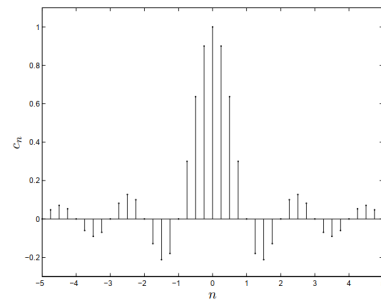


Fig. 22: Fourier coefficients of the function $\Pi(x)$ with period 4: the coefficients seem to be closer to each other

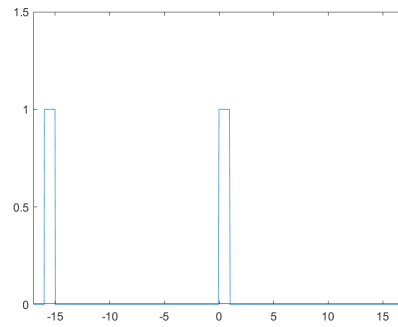


Fig. 23: The periodization of $\Pi(x)$ is further increased to 16.

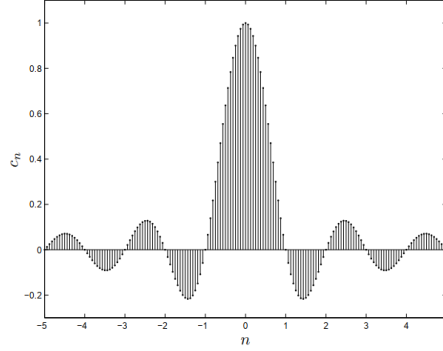


Fig. 24: Fourier coefficients of the function $\Pi(x)$ with period 16, the coefficients are closer and form a distinct waveform

The result is astonishing: in fact (as seen through Figure 20 to Figure 24), as the period grows, the proximity of the coefficients increases and the form of a wave is distinguishable.

Indeed, taking a careful look at the Fourier Series

$$f(x) = \sum_{n=-\infty}^{\infty} c_n e^{\frac{in\pi x}{T}}. \quad (22)$$

We can see that with the increase of T , the spectrum of frequencies is getting tighter, as the points are clustering closer and closer.

Evaluating the coefficients for the Fourier Series for the example of the $\Pi(x)$ function gives

$$\begin{aligned} c_n &= \frac{1}{T} \int_0^T e^{\frac{-2\pi int}{T}} f(t) dt = \frac{1}{T} \int_{-\frac{T}{2}}^{\frac{T}{2}} e^{\frac{-2\pi int}{T}} f(t) dt = \\ &= \frac{1}{T} \int_{-\frac{1}{2}}^{\frac{1}{2}} e^{\frac{-2\pi int}{T}} \cdot 1 dt = -\frac{1}{2\pi in} \left[e^{\frac{-2\pi int}{T}} \right]_{-\frac{1}{2}}^{\frac{1}{2}} = \\ &= -\frac{1}{2\pi in} \left[e^{-\frac{2\pi in}{T}} - e^{\frac{2\pi in}{T}} \right] = \frac{1}{2\pi in} \left[e^{\frac{2\pi in}{T}} - e^{-\frac{2\pi in}{T}} \right], \end{aligned}$$

which can be rewritten in the form:

$$\frac{1}{\pi n} \sin\left(\frac{\pi n}{T}\right).$$

Letting T tending towards infinity, $\frac{\pi n}{T}$ will approach 0, and as $\sin(\theta) = \theta$ for small θ also $\sin(\frac{\pi n}{T})$ will tend to 0. This means that having a non periodic function, our coefficients will get close to 0, which does not provide enough information. To have a clearer view of them, it is necessary to evaluate the

equation for $\frac{n}{T}$, so in this case to scale it up by T :

$$\Pi(t)(\text{periodized})\left(\frac{n}{T}\right) = T \frac{1}{\pi n} \sin\left(\frac{\pi n}{T}\right) = \frac{\sin(\pi n/T)}{\pi n/T}; \quad (23)$$

$(\frac{n}{T})$ should be condensed in a term s . Then the equation to compute the coefficient c_n should be reformulated

$$T \cdot c_n = T \cdot \frac{1}{T} \int_{-T/2}^{T/2} e^{-2\pi i s t} f(t) dt = \int_{-T/2}^{T/2} e^{-2\pi i s t} f(t) dt. \quad (24)$$

Hence, this is how the Fourier Transform is derived, and describes also the results seen in 24.

$$F(n/T) = \int_{-\infty}^{\infty} f(x) e^{-\frac{2\pi i n x}{T}} dx. \quad (25)$$

I found this graph (25) at [8], which gives, in my opinion, a vivid and concrete picture of a function in the time and in the frequency domain.

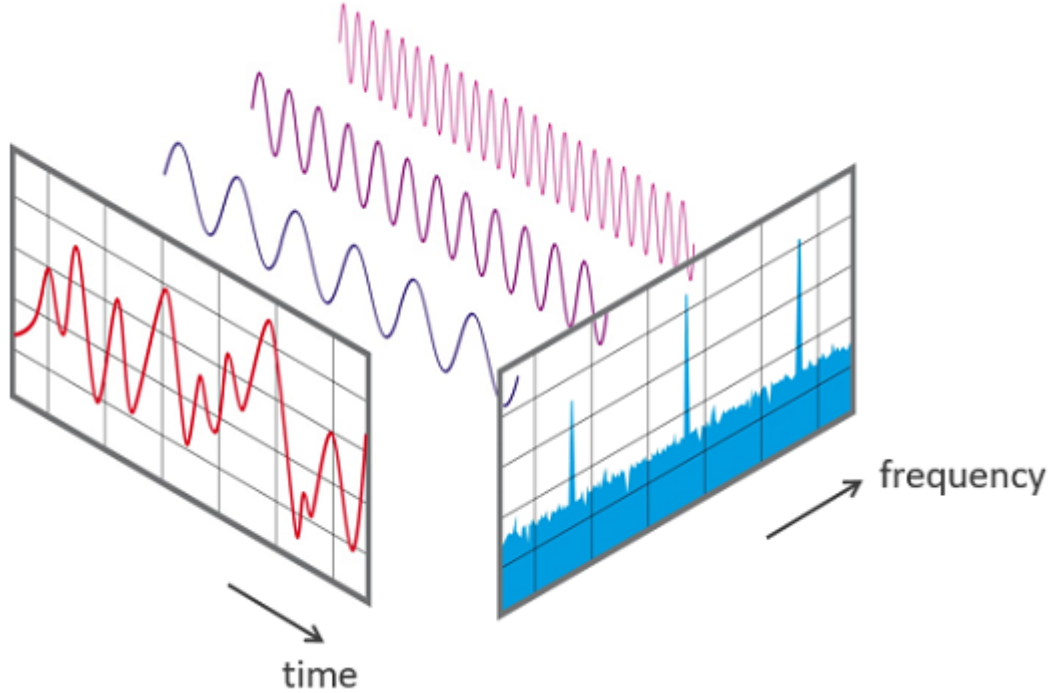


Fig. 25: signal in time domain and in frequency domain illustrated in a graph

3.3 Inverse Fourier Transform

It is also possible to recover the function in the spatial domain from the one in the frequency domain, and this is achieved through the Inverse Fourier Transform:

$$f(x) = \int_{-\infty}^{\infty} F(n/T) e^{\frac{2\pi i x n}{T}} dn/t. \quad (26)$$

3.4 Discrete Fourier Transform

Another flavour of the Fourier Transform is the Discrete Fourier Transform [14], which is given by mean of a Unitary Transform (Appendix B). This is a particular case of the Fourier transform in which a finite sequence of data is taking into consideration, while previously we worked with functions: it is of interest in image processing, as a digital image is formed by a finite number of pixels (See 3.1). The Discrete Fourier Transform $F[k, l]$ (with k and l being specific positions of a 2-dimensional sequence taken into consideration) of a sequence is given by

$$F[k, l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] e^{-j2\pi \left(\frac{k}{M} m + \frac{l}{N} n \right)}. \quad (27)$$

This could just be considered a "sampled" or "discrete" version of the Fourier Transform, and this is very useful in Digital Signal Processing.

If we want to travel from the Frequency Domain to the Spatial/Temporal domain (Inverse Discrete Fourier Transform) we can just proceed in this way:

multiply both sides of the equation by $e^{j2\pi \left(\frac{k}{M} m' + \frac{l}{N} n' \right)}$ (m' and n' constant integers) and then sum from $k = 0$ to $M - 1$ and from $l = 0$ to $N - 1$, with

the purpose of retrieving the sequence in the spatial domain:

$$\begin{aligned}
\sum_{k=0}^{M-1} \sum_{l=0}^{N-1} F[k, l] e^{j2\pi \left(\frac{k}{M} m' + \frac{l}{N} n' \right)} &= \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} e^{j2\pi \left(\frac{k}{M} m' + \frac{l}{N} n' \right)} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] e^{-j2\pi \left(\frac{k}{M} m + \frac{l}{N} n \right)} \leftrightarrow \\
\sum_{k=0}^{M-1} \sum_{l=0}^{N-1} F[k, l] e^{j2\pi \left(\frac{k}{M} m' + \frac{l}{N} n' \right)} &= \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} e^{-j2\pi \left(\frac{k}{M} m + \frac{l}{N} n \right) + j2\pi \left(\frac{k}{M} m' + \frac{l}{N} n' \right)} \leftrightarrow \\
&= \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} e^{\frac{j2\pi k}{M} (m' - m) + \frac{j2\pi l}{N} (n' - n)}, \\
e^{\frac{j2\pi k}{M} (m' - m) + \frac{j2\pi l}{N} (n' - n)} &= \begin{cases} 1, & m' = m \text{ and } n' = n \\ 0, & \text{otherwise} \end{cases}
\end{aligned}$$

This will result in $\sum_{k=0}^{M-1} \sum_{l=0}^{N-1} e^{\frac{j2\pi k}{M} (m' - m) + \frac{j2\pi l}{N} (n' - n)}$ to be $M \cdot N$ when $m' = m$ and $n' = n$:

$$\sum_{k=0}^{M-1} \sum_{l=0}^{N-1} F[k, l] e^{j2\pi \left(\frac{k}{M} m + \frac{l}{N} n \right)} = M \cdot N \cdot f[m, n] \rightarrow \quad (28)$$

$$f[m, n] = \frac{1}{M \cdot N} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} F[k, l] e^{j2\pi \left(\frac{k}{M} m + \frac{l}{N} n \right)}.$$

But in this way the Transform is not unitary, because the scale factor is not consistent in the forward and inverse transform, so last but not least, it is necessary to normalize it with the scale factor $\frac{1}{\sqrt{N \cdot M}}$

Hence, endly we can say that the DFT takes the form:

$$F[k, l] = \frac{1}{\sqrt{M \cdot N}} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} f[m, n] e^{-j2\pi \left(\frac{k}{M} m + \frac{l}{N} n \right)}. \quad (29)$$

And the inverse DFT:

$$f[m, n] = \frac{1}{\sqrt{M \cdot N}} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} F[k, l] e^{j2\pi \left(\frac{k}{M} m + \frac{l}{N} n \right)}. \quad (30)$$

Now the basic knowledge needed to understand image sampling is established, so we can continue.

3.5 Sampling

As theoretically the image function $f(x, y)$ is continuous, defined for each real number x and y , and the available space on digital storage is limited, it is necessary to discretize it through the processes of *sampling* and *quantization*. It is vital that these processes are applied in a careful and accurate manner, as mistakes or imprecisions at this stage could affect a lot the result: in particular a problem that should be dealt with is aliasing distortion, which will be discussed later. To achieve the discreteness of the image function, the idea is to have some impulses that capture the function value at specific finite points.

A convenient way to represent the sampled function $f_s(x, y)$ is to express it as the product of $f(x, y)$ times a sampling function $s(x, y)$:

$$f_s(x, y) = f(x, y)_{x=m\Delta x, y=n\Delta y} = f(x, y) \cdot s(x, y), \quad (31)$$

where m and n are positive integers, while Δx and Δy describe the spacing of the samples. Intuitively, increasing the spacing will lead to deterioration of quality, while decreasing it will improve it.

It is now introduced the concept of the impulse *Dirac Delta function*, which is later used to sample

$$\begin{aligned} \delta(x, y) &= 0 \quad x, y \neq 0, \\ \lim_{\epsilon \rightarrow 0} \int_{-\epsilon}^{\epsilon} \delta(x, y) dx dy &= 1. \end{aligned} \quad (32)$$

An important property of the Dirac Delta function is the *sifting property*.

Theorem 2 (Sifting Property). *As the Dirac Delta function is 0 whenever $x, y \neq 0$, the integral of the product between the Dirac Delta and another function, $f(x)$ is equal to the integral of $\delta(x) \cdot f(0)$, with $f(0)$ being a constant (as described in [9]):*

$$\begin{aligned} \int_{-\infty}^{\infty} \delta(x) \cdot f(x) dx &= \int_{-\infty}^{\infty} \delta(x) \cdot f(0) dx = \\ &= f(0) \int_{-\infty}^{\infty} \delta(x) dx = \\ &= f(0) \cdot 1 = f(0). \end{aligned}$$

The equality can be generalized furthermore as

$$\begin{aligned} \int_a^b \delta(x) \cdot f(x) dx &= \int_{-\infty}^{\infty} \delta(x) \cdot f(0) dx \quad a < b = \\ &= f(0) \int_a^b \delta(x) dx = \\ &= \begin{cases} f(0), & a < 0 < b \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

And if we take into consideration $\delta(x-\lambda)$ it can be observed that the function is always 0, except when x is equal to λ

$$\int_a^b \delta(x-\lambda) \cdot f(x) dt = \begin{cases} f(\lambda), & a < \lambda < b \\ 0, & \text{otherwise.} \end{cases} \quad (33)$$

The sampling function is then summarized as follows

$$s(x, y) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \delta(x - m\Delta x, y - n\Delta y). \quad (34)$$

The sampling function will basically ignore all function value at points other than multiples of Δx and Δy . Hence the sampled function $f_s(x, y)$ writes as

$$f_s(x, y) = f(x, y) \cdot s(x, y) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} f(m\Delta x, n\Delta y) \delta(x - m\Delta x, y - n\Delta y). \quad (35)$$

It is decisive to choose appropriate sample sizes for Δx and Δy , otherwise it can happen to stumble into problems in the signal reconstruction from the samples, like aliasing. The choice of sample sizes should be decided based on the frequency domain, therefore now it is the adequate time to use the previous discussed Fourier Transform (3.2).

The Fourier Transform of $f(x, y)$ is

$$F(\omega_x, \omega_y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-2\pi(\omega_x x + \omega_y y)} dx dy, \quad (36)$$

which entail two variables: the difference with the 1-dimensional transform is in the further multiplication and integration with respect to the second variable. And the Fourier Transform of $s(x, y)$ is proved to be ([14])

$$FT\{s(x, y)\} = \frac{1}{\Delta x \Delta y} \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \delta(\omega_x - m\omega_{xS}, \omega_y - n\omega_{yS}) dx dy. \quad (37)$$

Given the Convolution Theorem [5]

Theorem 3 (Convolution Theorem). $F(f(x) \otimes g(x)) = f(x)g(x)$,

or

$$F[f(x)g(x)] = F(f(x)) \otimes F(g(x)).$$

In words, multiplying two functions in the spatial or temporal domain is equivalent to take the Fourier Transform of their convolution, where The convolution operator is defined as

$$(f \otimes g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau. \quad (38)$$

As in the space domain the sampled function takes the form as in Eq (35), applying Theorem 3, just mentioned above, we can reformulate the equation in the frequency domain as follows

$$\begin{aligned}
F_s(\omega_x, \omega_y) &= F(\omega_x, \omega_y) \otimes FT\{s(x, y)\} = \\
&= \int_{-\infty}^{\infty} \left(F(\tau_x, \tau_y) \frac{1}{\Delta x \Delta y} \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \delta(\omega_x - \tau_x - m\omega_{xS}, \omega_y - \tau_y - n\omega_{yS}) \right) d\omega_x d\omega_y.
\end{aligned} \tag{39}$$

Because of the sifting property of the Dirac Delta function in Theorem 2:

$$F_s(\omega_x, \omega_y) = \frac{1}{\Delta x \Delta y} \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} F(\omega_x - m\omega_{xS}, \omega_y - n\omega_{yS}).$$

The recovery of the original image from the sampled one is possible only if the following conditions hold:

- a signal is band-limited, so in the case in which the Fourier Transform of the signal is 0 for the values outside of a range $[minFreq, maxFreq]$ and
- the value of $\frac{1}{\Delta x \Delta y}$ should not exceed a specific value, described below.

In the book [5] I think there is a very concrete visual explanation about this:

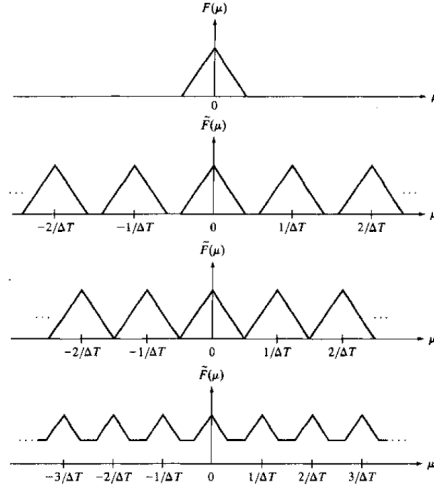


Fig. 26: The various periods by changing the $\Delta x \Delta y$ value: by decreasing ΔT , which in our case is $\Delta x \Delta y$, the period tends to overlap and it will become impossible to reconstruct the original signal

The limit spacing values to consider in order to reconstruct the original signal are described in the *Sampling Theorem* [14].

Theorem 4 (Sampling Theorem). *The Sampling Theorem states that, if a signal is band-limited at $[-\omega, \omega]$, then it is recoverable by uniform samples with a specific sample rate*

$$f_s \geq 2\omega,$$

or, considering the sampling period, the following hold

$$T \geq \frac{1}{2\omega}.$$

If the condition is complied with, it is possible to recover the original signal.

This is accomplished through the multiplication of the sampled signal by an ideal low-pass filter $H(\omega_x, \omega_y)$, defined as

$$H(\omega_x, \omega_y) = \begin{cases} \Delta x \Delta y, & |\omega_x| \leq \text{minFreq}, |\omega_y| \leq \text{maxFreq} \\ 0, & \text{otherwise.} \end{cases} \quad (40)$$

This can be derived from the visual explanation given in Figure 26. As explained above, multiplying the sampled frequency signal by $H(\omega_x, \omega_y)$ would mean recovering the original signal

$$F'(\omega_x, \omega_y) = F_s(\omega_x, \omega_y) \cdot H(\omega_x, \omega_y) = F(\omega_x, \omega_y);$$

and, for the Convolution Theorem 3, this is equivalent to convolute the two functions in the space domain

$$f'(x, y) = f_s(x, y) \otimes h(x, y) = f(x, y),$$

which yields

$$f'(x, y) = f(x, y) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} f(m\Delta x, n\Delta y) \left(\frac{\sin(\pi(xF_{xS} - m))}{\pi(xF_{xS} - m)} \right) \left(\frac{\sin(\pi(yF_{yS} - n))}{\pi(yF_{yS} - n)} \right). \quad (41)$$

3.5.1 Aliasing

If the Sampling Theorem 4 limits are not respected, we slip into the problem of aliasing.

Due to inadequate sampling, we are not able to recover the continuous signal uniquely.

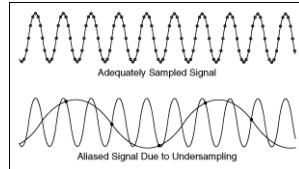


Fig. 27: The meaning of aliasing in a graph

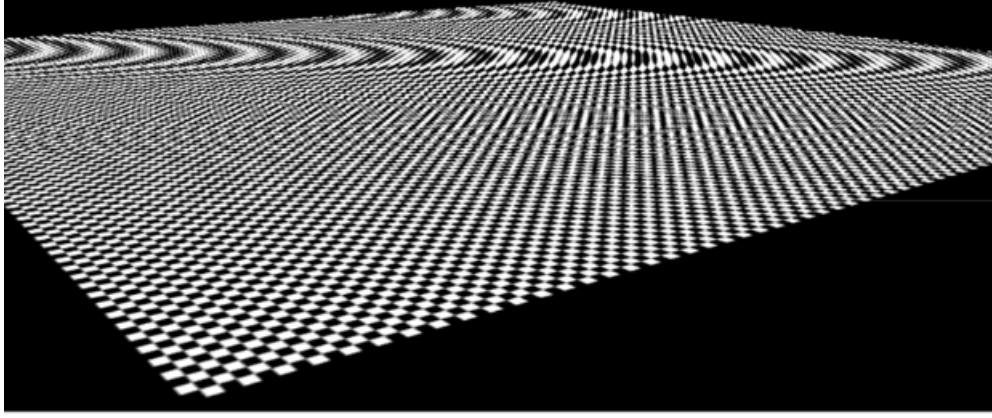


Fig. 28: Aliased image due to the lack of samples [15].

In Figure 27 it can be observed that the reconstructed signal is interpreted as a different version of the original one, and this results in an inexact outcome.

In an image the effect could be the appearance of Moire patterns, which is a kind of interference as can be seen in Figure 28.

3.6 Quantization

In the previous section we discussed about sampling, which is strictly related to the x-axis of the signal, while now it is necessary to introduce quantization, which deals with the y-axis of the signal. As digital storage cannot contain an infinite range of values, it is necessary to adjust the amplitudes of the signal into discrete values.

So, it can be said that the image has k levels, based on how vast is the range of values used to discretize the intensities. In the digital environment we are working with bits. Therefore, based on how many bits we use, we have different accuracy in quantization: the more bits we have, the more accurate the quantization will be.

The amount of levels is given by this formula

$$L = 2^k; \quad (42)$$

and, equivalently, the number of bits needed to address an output level is

$$k = \lceil \log_2 L \rceil \text{bits}. \quad (43)$$

In order to quantify a signal, it is necessary to map an input to an output, and this is decided based on some specific rules. The decisions are taken considering the fact that we want to build an optimal quantizer, which is in charge of satisfying the minimization of some criterion.

3.6.1 Lloyd-Max Quantizer

The Lloyd-Max Quantizer depends on the *Probability Density Function* (PDF) of the input sample, which is treated as a random variable.

Given a random variable x , which represents our sampled signal, which has a continuous pdf $p_x(x)$, we must map the input decision intervals d_k to the corresponding reconstruction levels r_k for L levels, such that the *Mean Square Error* is minimized. The Mean Square Error (MSE) of an estimator x with respect to an estimated parameter $\hat{x}$ is defined as

$$MSE = E[(x - \hat{x})^2] = \int_{d_1}^{d_{L+1}} (x - \hat{x})^2 p_x(x) dx, \quad (44)$$

which can be expressed as a sum of levels from level $j = 1$ to the last L^{th} level, as the integration in each range $[d_j, d_{j+1})$ is equal to a constant reconstruction level r_j . Hence, we rewrite it in this way:

$$MSE = \sum_{j=1}^L \int_{d_j}^{d_{j+1}} (x - r_j)^2 p_x(x) dx.$$

To compute the minimization of the MSE, it is needed to have a derivative with respect to d_j and one with respect to r_j and put them equal to 0 to find their minimum:

$$\begin{aligned} \frac{\delta MSE}{\delta d_j} &= (d_j - r_{j-1})^2 p_x(d_j) - (d_j - r_j)^2 p_x(d_j) = 0 \leftrightarrow \\ &\leftrightarrow = (d_j - r_{j-1})^2 = (d_j - r_j)^2 \leftrightarrow \\ &\leftrightarrow = |d_j - r_{j-1}| = |d_j - r_j|. \end{aligned}$$

Since r_j is greater than r_{j-1} , so, simplifying we obtain

$$d_j = \frac{r_j + r_{j-1}}{2}. \quad (45)$$

We can see from Eq. (45) that the optimal decision intervals are located in the middle of the considered output levels. On the other hand, putting the derivative with respect to r_s equal to 0

$$\frac{\delta MSE}{\delta r_j} = 2 \int_{d_j}^{d_{j+1}} (x - r_j) p_x(x) dx = 0, \quad 1 \leq j \leq L$$

gives

$$r_j = \frac{\int_{d_j}^{d_{j+1}} x p_x(x) dx}{\int_{d_j}^{d_{j+1}} p_x(x) dx}.$$

Here, instead we are considering the optimal output level, and they seem to be found at the mean position of the decision intervals.

3.7 Filtering

The first part was meant to provide a background on how the images are stored in digital form from an analog signal, now a step further is taken to dive into some Image Processing techniques, in particular filtering. Through some filters it is possible to modify an image to extract different information from an image.

We can distinguish between *low-pass* and *high-pass* filters.

A low-pass filter applied emphasizes the low frequencies, attenuating the high ones, resulting in a smoothed image result. The low-frequencies constitute the most basic features of an image. In Figure 29 we see an abstraction of the low-pass filtering process.

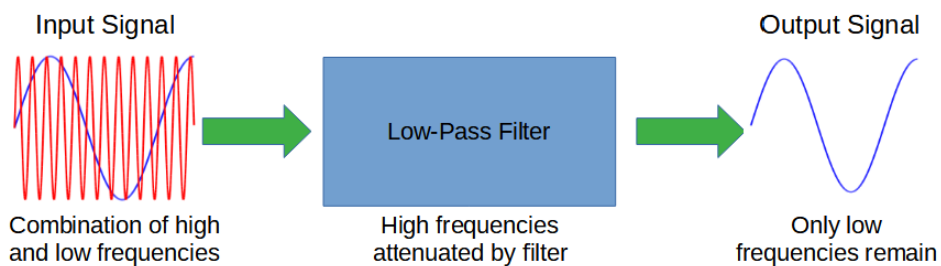


Fig. 29: A Low-Pass filter emphasizes the low frequencies of an image and attenuates its high frequency components

On the other hand, a high-pass filter can be used to extract the details of an image and for edge detection, as the high-frequency are the detail components of the image.



Fig. 30: Example of high-pass filter applied to an image: the edges of the objects are emphasized.

Filtering can be applied both in the spatial and in the frequency domain.

3.7.1 Filtering in Spatial Domain

Given a filter in matrix form, called also kernel or mask, filtering in the Spatial Domain is achieved through *correlation* or *convolution* between the image and filter. The processes of correlation and convolution are very similar, and they are neighborhood operations, meaning that, by applying correlation or convolution to an image, each pixel is influenced by its neighbours and the operations applied to them. Now the steps to achieve filtering will be described:

- place the kernel (filter) so that its rightmost and down element correspond to the origin of the image, and, if necessary, pad the left with zeros
- multiply each element of the image by each element of the mask;
- sum up all the elements found;
- move the kernel forward by one step;
- repeat, until reaching the end of the matrix;

- move the kernel down by one step;
- repeat the previous steps.

What should be pointed out, is that the result values strongly depend on the filter and on the values of neighbour pixels, since a linear combination is performed. The only difference between correlation and convolution is that in convolution the mask is rotated by 180 degrees.

Some MATLAB examples Here I give an example of a script which filters a moon image, firstly with a low-pass filter, secondly with a high-pass filter.

Listing 5: Script that filters an image with a low-pass filter and then with a high-pass filter in the space domain

```

1  clc; clear; close all;
2
3  img = rgb2gray(imread('moon.jpg'));
4
5  %show image
6  figure;
7  imshow(img);title('original image');
8  figure;
9  imhist(img);title('image intensities');
10
11 %frequency of image thorough fourier transform
12 fImg = fftshift(log(abs(fft2(img))));
13 figure;
14 imshow(fImg, []);title('fourier transform of original image');
15
16 %low pass filter in space domain
17 lowPass = imgaussfilt(img, 10);
18 figure;
19 imshow(lowPass);title('low-pass filtered image');
20 figure;
21 imhist(lowPass);title('low-pass filtered image intensities');
22 fImgLow = fftshift(log(abs(fft2(lowPass))));
23 figure;
24 imshow(fImgLow, []);title('fourier transform of low-pass
    filtered image');
25
26 %high pass filter in space domain
27 hHighPass = [1 0 -1; 1 0 -1; 1 0 -1];
28 highPass = imfilter(img,hHighPass);
29 figure;

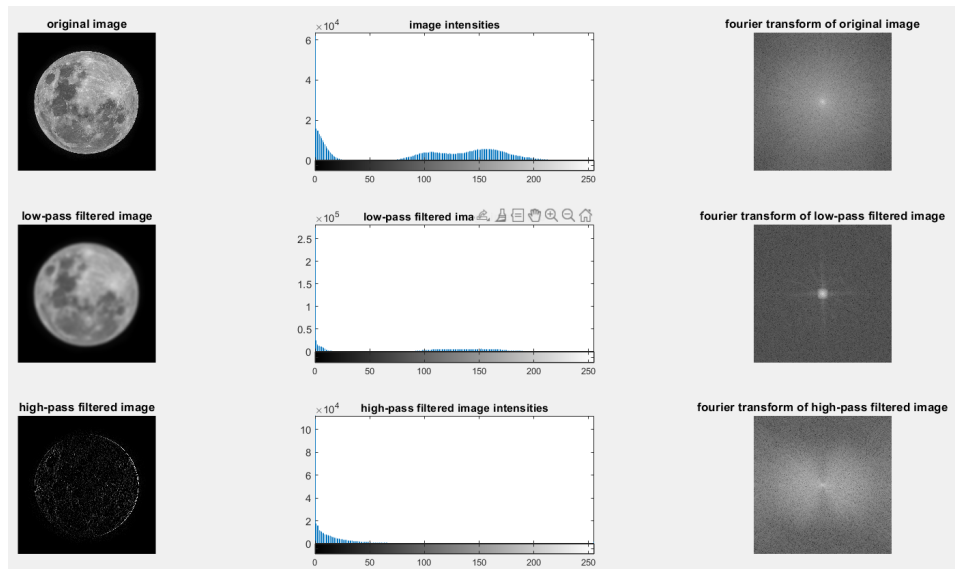
```

```

30 imshow(highPass);title('high-pass filtered image');
31 figure;
32 imhist(highPass);title('high-pass filtered image intensities');
33 fImgHigh = fftshift(log(abs(fft2(highPass))));
34 figure;
35 imshow(fImgHigh, []);title('fourier transform of high-pass
    filtered image');

```

And the result can be seen here:



The first row represents the original image, with its pixels' intensities, and its Fourier Transforms. The third row contains the high-pass filtered image. On the other hand, in the second row we are seeing the same image after applying a low-pass filter, its respective pixels' intensities and its Fourier Transform. The filter that I am using to cut high-frequencies is the *Gaussian filter*, which represents the shape of a Gaussian function (bell-shaped): as can be seen in Figure 31 the high-frequencies are cut smoothly, as the function has gentle slopes.

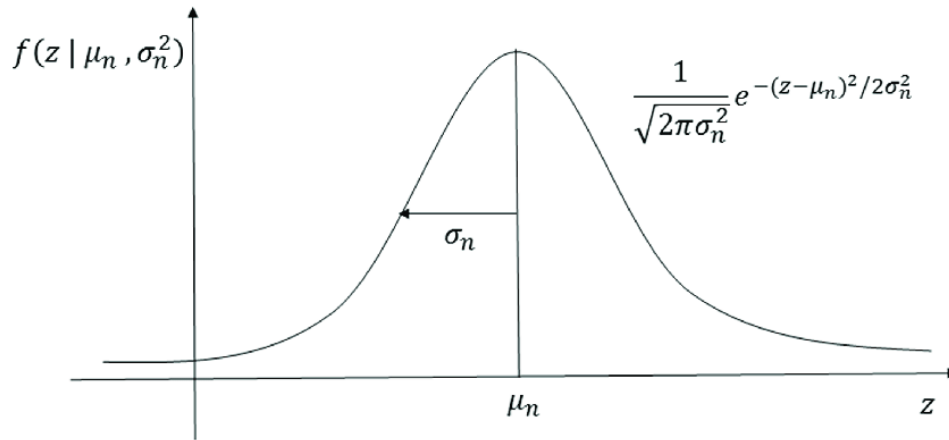


Fig. 31: Gaussian function.

3.7.2 Filtering in Frequency Domain

Sometimes it is more convenient to apply the filters in the frequency domain (for example in the case of finding some noise in the signal), and given the Convolution Theorem, as in spatial domain it is applied through convolution, in this case we must multiply the frequency domain image function by a frequency domain filter. In [10] I encountered a very useful and practical approach to the topic: there are some examples about low-pass, high-pass and notch filters applied in the Frequency Domain.

While low-pass filters and high-pass filters were already seen above, notch filters must be explained. Notch filters' job is to remove or attenuate some specific frequencies range; they are really convenient in removing some frequency-noise in an image. They usually involve trying to understand which frequencies should be eliminated, as they could be different for each distorted image taken into consideration, so, the filter should be changed for each distorted image. Usually the frequencies to be cut are the bright peaks not at the origin, which represent some drastic variation in pixel intensities (hence, high-frequencies). We will see some examples in MATLAB, so to understand better the concept. I would find it useful to use the examples mentioned in the reference, but using the filters and the image given in the "Filtering in Spatial Domain" section.

Some MATLAB examples

Listing 6: Script that filters an image with a low-pass filter and then with a high-pass filter in frequency domain

```

1 clc; clear; close all;
2
3 img = rgb2gray(imread('moon.jpg'));
```

```

4
5 %show image
6 subplot(2,3,1);
7 imshow(img);title('original image');
8
9 %frequency of image thorough fourier transform
10 fImgOrig = fft2(img);
11 fImg = fftshift(log(abs(fImgOrig)));
12 subplot(2,3,2);
13 imshow(fImg, []);title('fourier transform of original image');
14
15
16 [fImgR, fImgC] = size(fImgOrig);
17
18 %low pass filter in frequency domain
19 hLowPass = fspecial('gaussian', fImgC, 7);
20 HLowPass = fft2(fftshift(hLowPass));
21 subplot(2,3,3);
22 imshow(log(fftshift(abs(HLowPass))),[]);title('low-pass filter
    frequency');
23
24 %apply filter in frequency domain
25 fLowPass = HLowPass .* fImgOrig;
26 subplot(2,3,4);
27 imshow(log(fftshift(abs(fLowPass))),[]);title('low-pass filtered
    image frequency');
28
29 sLowPass = real(ifft2(fLowPass));
30 subplot(2,3,5);
31 imshow(sLowPass,[]);title('low-pass filtered image shown');
32
33 figure;
34 subplot(2,3,1);
35 imshow(img);title('original image');
36
37 %frequency of image thorough fourier transform
38 fImgOrig = fft2(img);
39 fImg = fftshift(log(abs(fImgOrig)));
40 subplot(2,3,2);
41 imshow(fImg, []);title('fourier transform of original image');
42
43 %high pass filter in frequency domain
44 hHighPass = zeros(fImgR, fImgC);
45 hHighPass((fImgR/2)-1:(fImgR/2)+1, (fImgC/2)-1:(fImgC/2)+1) = [1

```

```

    0 -1; 1 0 -1; 1 0 -1];
46
47 HHighPass = fft2(fftshift(hHighPass));
48
49 HHighPass(1,1000) = 0;
50 HHighPass(1000,1) = 0;
51 subplot(2,3,3);
52 imshow(HHighPass,[]);title('high-pass filter frequency');
53
54 %apply filter in frequency domain
55 fHighPass = HHighPass .* fImgOrig;
56 subplot(2,3,4);
57 imshow(log(fftshift(abs(fHighPass))),[]);title('high-pass
    filtered image frequency');
58
59 sHighPass = real(ifft2(fHighPass));
60 subplot(2,3,5);
61 imshow(sHighPass,[]);title('high-pass filtered image shown');

```

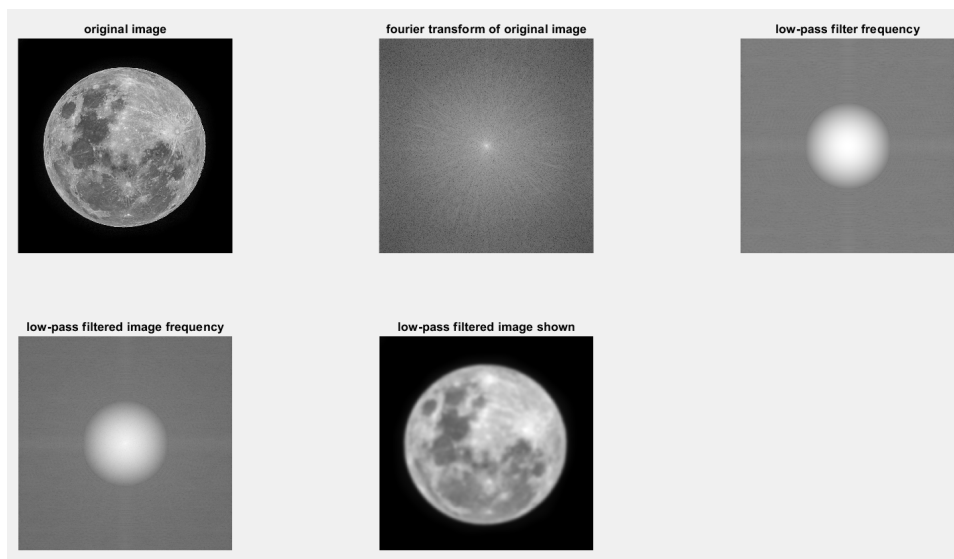


Fig. 32: Low pass filter applied to the image in frequency domain through matrix multiplication.

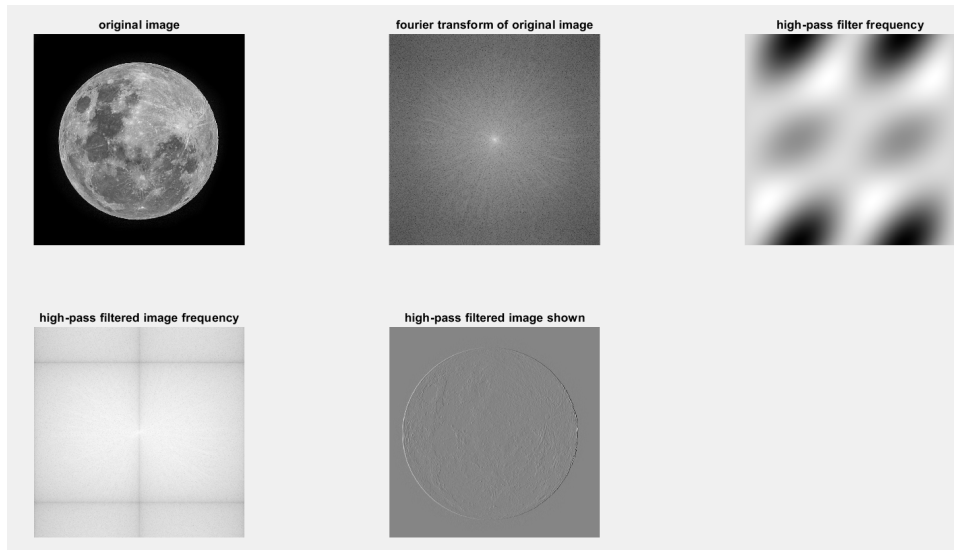


Fig. 33: High pass filter applied to the image in frequency domain through matrix multiplication.

4 Image Compression

We described some image processing techniques in the previous section, but another type of manipulation of great concern is image compression, which is used to spare storage space.

The challenge is to find a good compromise between high-quality and space occupied by images, that is why it is also useful to have some background also in how the human visual system works. In fact, for example, we could exploit the fact that we are more sensible to lightwaves in a for a specific range of frequencies, and attenuate the frequencies that are for us less distinguishable, so to save memory storage and not to prejudice quality.

We can divide compression into two different classes: lossless and lossy. The difference between them, as the name suggests, is that the latter is losing some data during the process, while in the first case the recovery of the original image is possible without any loss of information.

4.1 Lossless compression

As said previously, lossless compression foresee no loss of data. Usually manipulation of neighbouring pixel values is used, also through statistical methods.

One popular lossless compression methods is called Huffman Coding, and it is now introduced.

4.1.1 Huffman Coding

Huffman Coding is a lossless compression algorithm based on the concept of entropy of the image data. But what is entropy?

Entropy Entropy is defined to be the expected quantity of information a signal carries, and it is based on the probability theory. The more probable an event is to occur, the less information it contains. In fact, if we take a simple example, let's consider the case in which someone says that tomorrow the sun is rising: the probability that this event will happen is basically 1, thus it does not contain much information, because everyone is already aware of this and has almost no doubt that it will happen. On the other hand, if someone is telling you that a meteorite hits the earth (assuming that this is a true information), then this event contains a very high amount of information, as it has a low probability to happen. Being informal, entropy can be regarded as the "degree of surprise" arisen by an event.

Firstly, the representation of information will be specified as follows:

Definition (Information). *By information, we define the following quantity:*

$$I(X = x_i) = \log_2 \left(\frac{1}{p_X(x_i)} \right), \quad (46)$$

given that $p_X(x_i)$ is the probability that X equals x_i , and the bit is the unit of measurement ($\log_2$). To have a straightforward example, consider a coin, which, being tossed, can take the values head or tail, with probability $p_X(\text{Head}) = 0.5$ and $p_X(\text{Tail}) = 0.5$. Then, in either cases one gets a Head or a Tail, one will receive the same amount of information, in particular:

$$I(X = \text{Head}) = \log_2 \left(\frac{1}{p_X(\text{Head})} \right) = \log_2 \left(\frac{1}{0.5} \right) = 1\text{bit},$$

and similarly for $I(X = \text{Tail})$.

Another example could be the one of a card deck composed by 52 cards, containing 4 different suits, each formed by 13 cards. If I draw a card, and I tell you that it is a diamond card (the card is a diamond with a probability of $\frac{13}{52}$), then the amount of information is

$$I(X = \text{Diamond}) = \log_2 \left(\frac{52}{13} \right) = 2 \text{ bits}.$$

At this point, we are ready to give the formal definition of entropy.

Definition (Entropy). *Considering a random variable X , which can take any value from a set of possible symbols $(x_1, x_2, x_3, \dots, x_n)$ with probabilities*

$p_X(x_1), p_X(x_2), p_X(x_3), \dots, p_X(x_n)$, the entropy $H(X)$ of X is defined as the expected value of the information given by the outcome of X :

$$H(S) = \sum_{i=1}^N p_X(x_i) I(X = x_i) = \sum_{i=1}^N p_X(x_i) \log_2 \left(\frac{1}{p_X(x_i)} \right). \quad (47)$$

The concept of entropy is really useful in information theory, as it provides information about the average amount of data needed to resolve the uncertainty about the outcome of a trial, which could produce a lower bound to the number of binary digits utilized in message encoding.

In [16] we can read the story of the discovery of Dave Huffman: his Professor, Robert M. Fano, assigned to his class the exercise to determine an efficient way to represent symbols using binary code. After months of attempts to find a solution, Huffman was about to give up, when he came up with the answer, and succeeded where his Professor and other important people struggled. The idea in Huffman Coding is to build a code tree based on the entropy $H(X)$ of a signal X , with the length of the code being L , with the constraint that $L \geq H(S)$.

Huffman found the minimal expected length code, which is achieved through the following ingredients:

- a set X of symbols $x_1, x_2, \dots, x_n$;
- the probability $p(x)$ for each symbol x in X ;

and the following steps:

- find the elements of X with the lowest probabilities;
- remove those elements from X , creating a new node whose children will be the elements just removed. The left branch will be assigned a 0, while the right one will be a 1;
- a new symbol corresponding to the new node will be added to X , and it will be assigned the value of the sum of the probabilities of its children;
- repeat this procedure until only one symbol is left in X .

The least common symbols to be found will be for sure the one to have a longer binary code, since they are the first ones to be removed from the alphabet, and they will be the leaves of the created tree, so the nodes with the longest traversal path. On the other hand, the most probable symbols are to be seen in the first levels of the tree, as they are the last elements to be chosen from the set. Let's take a look at a simple example. Let's consider a set $X = \{ 'a', 'b', 'c', 'd' \}$, with the following probabilities $p_X(a) = \frac{1}{2}$, $p_X(b) = \frac{1}{8}$, $p_X(c) = \frac{1}{4}$, $p_X(d) = \frac{1}{8}$.

- The two symbols with the lowest probabilities are b and d , so we will remove them from X . Now $X = \{a, c\}$. b is 0, d is 1. The parent of b and d is a new node $n1$.
- We add $n1$ to X : $X = \{a, c, n1\}$, and $p_X(n1) = p_X(b) + p_X(d) = \frac{1}{4}$.
- c and $n1$ are the least probability valued elements, so we remove them from X , and add them as children of a new node $n2$. c is 0 and $n1$ is 1.
- $n2$ is added to X , and it is assigned probability $p_X(n2) = \frac{1}{2}$.
- We eliminate a and $n2$ from X ; a is 0, $n2$ is 1. We create a new node $n3$, which is the parent of a and $n2$.
- We add $n3$ to X , with probability $p_X(n3) = 1$.
- We are finished, as there is only one element left in X .

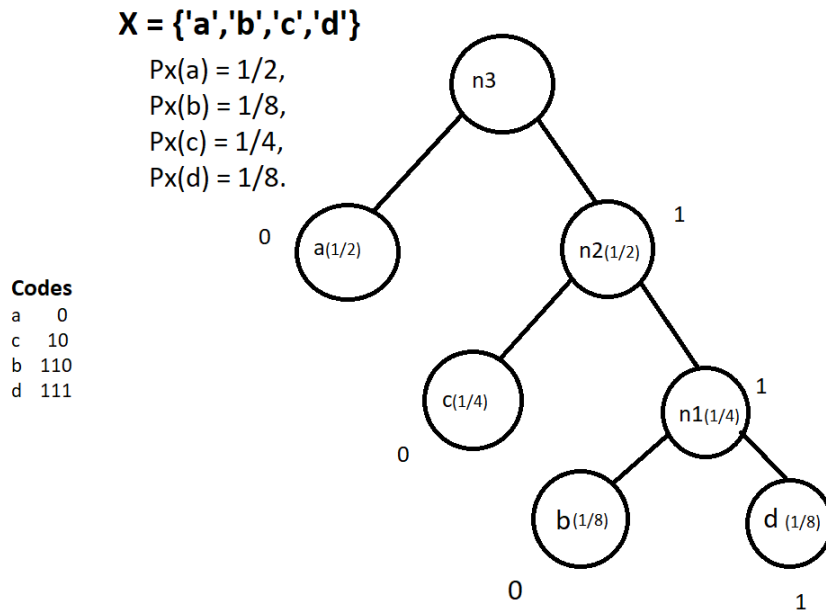


Fig. 34: The representation of the example: it can be noticed that the symbol with the highest probability (*a*) is the one with the shortest path.

A implementation of the Huffman algorithm applied to an image in MATLAB. I must admit that it was not so trivial to code this part.

Listing 7: Script which build the Huffman tree given an image

```

1 clear; close all;
2
3 img = rgb2gray(imread('input.png'));
4 [r,c] = size(img);
5
6 %find number of pixels which have specific intensity
7 [numPixel, binLocations] = imhist(img);
8 [sizePixels,~] = size(numPixel);
9
10 %remove all zero values

```

```

11 for i = 1:sizePixels
12     if(numPixel(i,1) == 0)
13         binLocations(i,1) = 0;
14     end
15 end
16 numPixel = numPixel(numPixel ~= 0);
17 binLocations = binLocations(binLocations ~= 0);
18 [sizePixels,~] = size(numPixel);
19
20 %find probability of each intensity
21 probabilities(1,:) = double(double(numPixel) ./ double(r*c));
22
23 %initialize node structure for huffman tree
24 node.intensity = 0.0;
25 node.probability = 0.0;
26 node.code = NaN;
27 node.parent = 0;
28 node.child1 = 0;
29 node.child2 = 0;
30 treeNodes = repmat(node,1,sizePixels);
31
32 %initialize tree in unsorted order
33 for i = 1:sizePixels
34     node.intensity = binLocations(i,1);
35     node.probability = probabilities(1,i);
36     treeNodes(1,i) = node;
37 end
38
39 %sort tree and apply huffman algorithm
40 lastIndex = sizePixels;
41 sizeIndex = sizePixels*2-2;
42 sortedtreeNodes = repmat(node,1,sizeIndex);
43 position = 1;
44 minimum1 = 0;
45 parentIntensity = -1;
46
47 while length(treeNodes) > 1
48     minProb1 = 1.1;
49     minProb2 = 1.1;
50
51 %find the two symbols which have the least probability
52     for i = 1:lastIndex
53         if(treeNodes(1,i).probability < minProb1)
54             minProb2 = minProb1;

```

```

55         minimum2 = minimum1;
56         minProb1 = treeNodes(1,i).probability;
57         minimum1 = i;
58         elseif(treeNodes(1,i).probability < minProb2 &&
59                 treeNodes(1,i).probability ~= minProb1)
60             minProb2 = treeNodes(1,i).probability;
61             minimum2 = i;
62         end
63     end
64     %assign code 0 to left sibling and 1 to right sibling
65     sortedtreeNodes(1,position) = treeNodes(1,minimum1);
66     sortedtreeNodes(1,position).code = 0;
67     sortedtreeNodes(1,position+1) = treeNodes(1,minimum2);
68     sortedtreeNodes(1,position+1).code = 1;
69
70     if(sortedtreeNodes(1,position).child1 ~= 0)
71         sortedtreeNodes(1, sortedtreeNodes(1,position).
72             child1).parent = position;
73     end
74
75     if(sortedtreeNodes(1,position+1).child1 ~= 0)
76         sortedtreeNodes(1, sortedtreeNodes(1,position+1).
77             child1).parent = position+1;
78     end
79
80
81     %parent node
82
83     treeNodes(minimum1).intensity = parentIntensity;
84     treeNodes(minimum1).probability = sortedtreeNodes(1,
85         position).probability + sortedtreeNodes(1,position
86         +1).probability;
87     treeNodes(minimum1).child1 = position;
88     treeNodes(minimum1).child2 = position+1;
89     parentIntensity = parentIntensity - 1;
90
91     treeNodes(minimum2) = [];
92     minProb1 = 1.1;
93     minProb2 = 1.1;

```

```

92
93         lastIndex = lastIndex-1;
94         position = position + 2;
95     end
96
97     sortedtreeNodes(1, position) = struct('intensity', -1, '
        probability', sortedtreeNodes(1,position-1).probability +
        sortedtreeNodes(1,position-2).probability,'code', 0, 'parent
        ', 0, 'child1', position-2, 'child2',position-1);
98     sortedtreeNodes(1,position-1).parent = position;
99     sortedtreeNodes(1,position-2).parent = position;
100
101     %get codes by traversing the tree
102     codes = traverseTree(sortedtreeNodes,position);
103
104     %write the image in code form
105     codeLengths = zeros(1, length(codes));
106     indexLengths = 1;
107     indexCodes = 1;
108     imageCodeArr = strings(r,1);
109     imageCode = ;
110     [binLocationsSize, ~] = size(binLocations);
111     for i = 1:binLocationsSize
112         fprintf(Intensity: %d -> %s\n, binLocations(i) ,codes(
            binLocations(i)));
113     end
114     for i = 1:r
115         for j = 1:c
116             imageCode = strcat(imageCode,codes(img(i,j)));
117             codeLengths(indexLengths) = length(codes(img(i,j)));
118             indexLengths = indexLengths + 1;
119         end
120
121         imageCodeArr(indexCodes, 1) = imageCode;
122         indexCodes = indexCodes + 1;
123         imageCode = ;
124     end
125     disp(imageCodeArr);

```

Listing 8: traverseTree.m function

```

1 function traverse = traverseTree(tree, start)
2     %node = start;
3     code = ;

```

```

4
5     result = containers.Map('KeyType','int32', 'ValueType','
        any');
6     node = start;
7     mapIndex = 1;
8     queue(1,1) = -inf;
9
10    %iterate the tree in stack fashion and get each element code
11    while ~isempty(queue)
12        while node ~= 0
13            if(tree(node).child2 ~= 0)
14                queue(end+1) = tree(node).child2;
15            end
16            if(tree(node).intensity ~= -1)
17                code = strcat(code, string(tree(node).code));
18            end
19            keys(mapIndex) = tree(node).intensity;
20            values(mapIndex) = code;
21            result = containers.Map(keys, values);
22            mapIndex = mapIndex + 1;
23            if(tree(node).child1 == 0)
24                prev = node;
25            end
26            fprintf('\nPosition: %d,node);
27            node = tree(node).child1;
28            fprintf('\nQueue: ');
29            disp(queue);
30        end
31
32        node = queue(end);
33        disp(code);
34        if(queue(end) ~= -Inf)
35            code = result(tree(tree(queue(end)).parent).
                intensity);
36        end
37        %code = "";
38        queue(end) = [];
39    end
40    traverse = result;
41 end

```

And the output looks like this:

```

Intensity: 72 -> 111111111
Intensity: 75 -> 111111110
Intensity: 79 -> 11011110
Intensity: 83 -> 0011110
Intensity: 84 -> 11011111
Intensity: 87 -> 1011110
Intensity: 89 -> 11110110
Intensity: 93 -> 0110110
Intensity: 94 -> 0011111
Intensity: 96 -> 000110
Intensity: 98 -> 100110
Intensity: 100 -> 11111110
Intensity: 102 -> 1101110
Intensity: 103 -> 001110
Intensity: 105 -> 101110
Intensity: 106 -> 1011111
Intensity: 110 -> 1111010
Intensity: 114 -> 011010
Intensity: 115 -> 11001
Intensity: 118 -> 00010
Intensity: 119 -> 10010
Intensity: 122 -> 111011
Intensity: 124 -> 111010
Intensity: 125 -> 01010
Intensity: 127 -> 01111
Intensity: 128 -> 1111110
Intensity: 130 -> 110110

```

Fig. 35: The table in which you can see the mapping between intensity of pixels and their code derived from the Huffman Tree.

The pair $(\lambda, \vec{x})$ is commonly referred to as an **Eigenpair** of matrix A .

All the possible distinct Eigenvalues are described by the **Spectrum of A** , denoted as $\sigma(A)$

Definition (Diagonalizable Matrix). A matrix A diagonalizable if there exists an invertible matrix P and a diagonal matrix D , such that

$$A = PDP^{-1}.$$

Equivalently, D can be expressed as:

$$D = P^{-1}AP.$$

If there exist such a representation of A , we can deduce that there are N distinct eigenvectors associated to A , and these are the columns of P . Meanwhile, A has also N distinct eigenvalues, which are the diagonal entries of D .

Definition (Symmetric Matrices). A matrix is defined as symmetric if $A^T = A$. An $N \times N$ symmetric matrix satisfies the following properties:

Properties.

- A has N real eigenvalues;
- the eigenvectors of A can be chosen with real entries;
- A has N linearly independent eigenvectors;
- A has N orthogonal eigenvectors;
- A has N orthonormal eigenvectors.

The third property means that A is a *diagonalizable matrix* (discussed in section 4.2.2).

4.2.3 Singular Value Decomposition

The main idea is to diagonalize a matrix, but, since often matrices are not square (in particular in an image context, we encounter often rectangular ones), we need to form a symmetric matrix from the one given. This is achieved by computing the products AA^T and $A^T A$.

To show that a matrix A , with dimensions $M \times N$ multiplied by its transpose A^T (which hence has dimensions $N \times M$ and is conformable to be multiplied by A) is symmetric, we must check that $AA^T = (AA^T)^T$. Recalling that

$(AB)^T = B^T A^T$ and that the transpose of the transpose of a matrix yields the matrix itself, the SVD decomposition of a rank- r matrix A writes as

$$(AA^T)^T = A(A^T) = AA^T,$$

and the same procedure can be used to show that $A^T A = (A^T A)^T$. So, shown this, it is convenient to represent how matrix A can be decomposed, considering that it has rank r [1]:

$$A \begin{bmatrix} v_1 & \dots & v_r \end{bmatrix} = \begin{bmatrix} u_1 & \dots & u_r \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \end{bmatrix},$$

where u 's are eigenvectors of AA^T , while v 's are eigenvectors of $A^T A$; the σ 's values are called the **singular values** of A .

Another equivalent form to write this is:

$$A = \begin{bmatrix} u_1 & \dots & u_r \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \end{bmatrix} \begin{bmatrix} v_1 & \dots & v_r \end{bmatrix}^T,$$

as V is a Orthonormal matrix. An important aspect is that the σ values appearing on the diagonal matrix are found to be ordered by importance: the first is the largest one, decreasing until the last, σ_r , which is the smallest one; mathematically $\sigma_1 > \sigma_2 > \dots > \sigma_r$.

What I omitted here is that under our r_{th} sigma a partition of zeros would be found, as we started working with a rectangular matrix, which contains some redundant values. Furthermore, SVD is bounded to the representation of images as an outer product of vectors: in fact, another way to picture the decomposition is to break down the three different components U , S , and V in rank 1 matrices:

$$A = U\Sigma V^T = u_1\sigma_1v_1^T + \dots + u_r\sigma_rv_r^T.$$

As the σ values are stored on a importance base, we know that the values found at the bottom of the matrix are the least significant ones. Consequently, we can think about setting a particular threshold, and discard all the values smaller than it. A point that I would regard as worth stressing on, is the intuitive meaning of the SVD. We are decomposing our initial matrix A in the three matrices U, S, V : U contains some basis vector, and their importance in the reconstruction of A is given by the σ value associated to it; the larger the sigma value, the more influence it has; on the other hand, the V matrix provides a description of the rate of change between the columns.

Now it would be interesting to check out some applications in the image field. I have prepared a script in MATLAB, which uses the `svd(img)` function to decompose the image into its U , V and Σ components and, it cut off all the values which are smaller than a specific threshold, encoding the image through Huffman Coding, and achieving in this way compression. I apply the SVD decomposition to each RGB channel separately, and then put them together again. In the example I firstly show the original image, then the one derived by the $U \times \Sigma \times V^T$ product; then I try to reduce the rank of the original image, by setting a specific threshold: in the first case I zero out all the elements smaller than 2, in the second case the ones less then 10, and I report the specific amount of dropped singular values. Then, using Huffman coding (4.1.1), some storage space can be spared.

Listing 9: SVD-based image compression in MATLAB

```

1 clear;
2 close all;
3
4 %read image
5 img = imread('dol.jpg');
6 figure;
7 imshow(img);
8
9 %separate the three different channels for different colours
10 imgCh1 = im2double(img(:,:,1));
11 imgCh2 = im2double(img(:,:,2));
12 imgCh3 = im2double(img(:,:,3));
13
14 [U1,S1,V1] = svd(imgCh1);
15 [U2,S2,V2] = svd(imgCh2);
16 [U3,S3,V3] = svd(imgCh3);
17
18 figure;
19 completeFigure(:,:,1) = U1*S1*V1';
20 completeFigure(:,:,2) = U2*S2*V2';
21 completeFigure(:,:,3) = U3*S3*V3';
22 imshow(completeFigure,[]);
23 threshold = 0.5;
24
25 S1(S1 < threshold) = 0;
26 i = 1;
27
28 while S1(i,i) ~= 0
29     i = i+1;
30 end

```

```

31
32 %count values being dropped
33 dropped = 0;
34 singularVal = i;
35 while (singularVal < min(size(S1)))
36     dropped = dropped + 1;
37     singularVal = singularVal + 1;
38 end
39 fprintf(Amount of dropped singular values: %d\n, dropped);
40 U1 = U1(:,1:i);
41 S1 = S1(1:i,1:i);
42 V1 = V1(:,1:i);
43 svdImg1 = U1*S1*V1';
44
45 figure;
46 imshow(svdImg1,[]);
47
48 S2(S2 < threshold) = 0;
49 i = 1;
50 while S1(i,i) ~= 0
51     i = i+1;
52 end
53 U2 = U2(:,1:i);
54 S2 = S2(1:i,1:i);
55 V2 = V2(:,1:i);
56 svdImg2 = U2*S2*V2';
57
58 figure;
59 imshow(svdImg2,[]);
60
61 S3(S3 < threshold) = 0;
62 i = 5;
63 while S1(i,i) ~= 0
64     i = i+1;
65 end
66 U3 = U3(:,1:i);
67 S3 = S3(1:i,1:i);
68 V3 = V3(:,1:i);
69 svdImg3 = U3*S3*V3';
70
71 figure;
72 imshow(svdImg3,[]);
73
74 totImg(:,:,1) = svdImg1;

```

```
75 totImg(:,:,2) = svdImg2;  
76 totImg(:,:,3) = svdImg3;  
77  
78 figure;  
79 imshow(totImg,[]);
```

And then I will present the result:



Fig. 37: The original image



Fig. 38: The image reconstructed from the product $U \times \Sigma \times V^T$



Fig. 39: Image compression by zeroing out the components < 2 (525 discarded singular values over 858, 61% of the values cut): the difference is not so noticeable from the original one



Fig. 40: Image compression by zeroing out the components < 10 (825 discarded singular values over 858, 96% of the values cut): the image's quality is not so good anymore.

To analyse the behaviour of the algorithm when changing the threshold, I applied the algorithm in loop, changing the threshold by 0.5 at each iteration, and then I plotted on the x-axis the threshold of the values, and on the y-axis the amount of dropped values (Figure 41). What can be noticed is that the function follows a logarithmic curve.

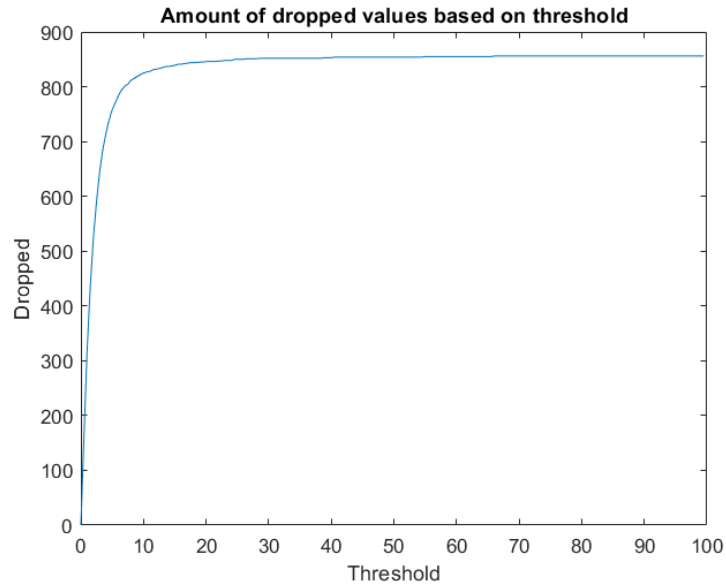


Fig. 41: Compression rate change by increasing threshold.

It would be interesting to plot how the image size is changing with respect to the compression ratio. Comparing the following plot (Figure 42) with Figure 41 it can be observed that the image size is strictly connected to the amount of values being dropped.

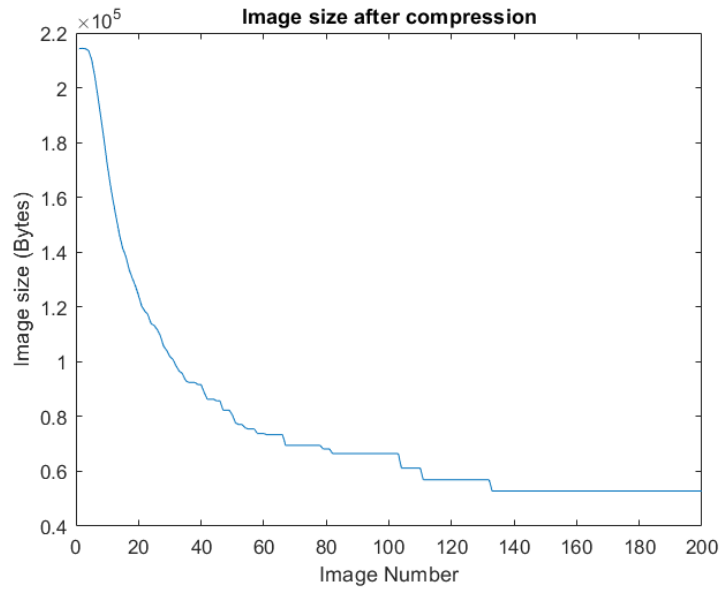


Fig. 42: The decrease of the size with the increased degradation of image quality.

4.2.4 JPEG

JPEG (Joint Photographic Experts Group) format uses the Discrete Cosine Transform to achieve image compression, which will be described now [14].

Discrete Cosine Transform Another Unitary-Transform based tool is the Discrete Cosine Transform, which in some cases can be more efficient than the Discrete Fourier Transform: in fact the DCT brings up some features of an image, therefore it is very useful to compress.

The DCT of a sequence $f[x, y]$, $0 \leq x \leq M-1; 0 \leq y \leq N-1$ can be derived from 3.2.1 and is described by:

$$F[k, l] = \alpha_k \alpha_l \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f[x, y] \cos\left(\frac{(2x+1)\pi k}{2M}\right) \cos\left(\frac{(2y+1)\pi l}{2N}\right), \quad 0 \leq k \leq M-1, 0 \leq l \leq N-1;$$

$$\alpha_k = \begin{cases} \frac{1}{\sqrt{M}}, & k = 0 \\ \sqrt{\frac{2}{M}}, & 1 \leq k \leq M-1; \end{cases}$$

$$\alpha_l = \begin{cases} \frac{1}{\sqrt{N}}, & l = 0 \\ \sqrt{\frac{2}{N}}, & 1 \leq l \leq N-1. \end{cases} \quad (49)$$

The DCT is orthogonal, so the original sequence can be recovered in this way

$$f[x, y] = \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} \alpha_k \alpha_l X[k, l] \cos\left(\frac{(2x+1)\pi k}{2M}\right) \cos\left(\frac{(2y+1)\pi l}{2N}\right). \quad (50)$$

Now the steps to achieve compression through JPEG will be described.

- Firstly, as the human visual system is more sensitive to intensity information than to colour information, it is necessary to convert the image from the RGB coordinate system to the YUV one (3.1.1).
- The chroma information contained in the UV may be downsampled, as the human visual system is less sensitive to UV info than to Y intensity data.
- Then, it is needed to divide the image data into 8 by 8 pixel blocks, and apply the Discrete Cosines Transform (4.2.4) to each block, jumping in this way from the spatial domain to the frequency domain. The output of this stage is a set of coefficients about spatial frequencies:

high frequency elements are located in the lower right part of the DCT block and are often neglectable, as the human visual system is not so sensitive to them, and they can be discarded, based on how much compression you want to achieve. Then, using Huffman Coding, it is possible to represent those zero values by a short code.

Now a MATLAB script which computes the various steps of JPEG for compressing an image will be shown as an example.

Listing 10: Simple JPEG script

```

1 clear;close all;
2
3 %read image and show it
4 img = imread('landscape.tiff');
5 figure; imshow(img);
6
7 %convert rgb to yuv
8 imgYUV = rgb2ycbcr(img);
9 [r,c,s] = size(imgYUV);
10
11 %subsample colors: human eye is more sensitive to intensity then
    chroma
12 imgYUV(:, :, 2) = imresize(imgYUV(1:30:r,1:30:c,2),[r,c]);
13 imgYUV(:, :, 3) = imresize(imgYUV(1:30:r,1:30:c,3),[r,c]);
14
15 %show the Y, Cb and Cr components
16 lb = {'Y', 'Cb', 'Cr'};
17 figure;
18 for channel = 1:3
19     subplot(1,3,channel)
20     Y_C = imgYUV;
21     Y_C(:, :, setdiff(1:3,channel)) = intmax(class(Y_C))/2;
22     imshow(ycbcr2rgb(Y_C))
23     title([lb{channel} ' component'],'fontsize',16)
24 end
25
26 %divide data into 8x8 blocks and apply algorithm to each of them
27 y = zeros(r,c);
28 block = zeros(8,8);
29
30 for i = 1:8:r-7
31     for j = 1:8:c-7
32         for channel = 1:3
33             block = dct(double(imgYUV(i:i+7,j:j+7,channel)));

```

```

34         block(4:8,4:8) = 0;
35         y(i:i+7,j:j+7,channel) = idct(block);
36     end
37 end
38 end
39
40 y = uint8(y);
41
42 resultImg = ycbcr2rgb(y);
43 figure; imshow(resultImg,[]);

```



Fig. 43: Input image

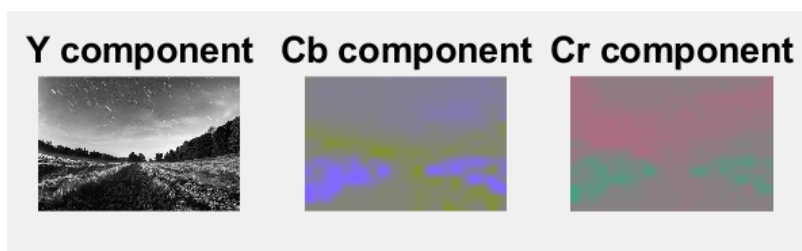


Fig. 44: The Y intensity component and the Cb, Cr chroma components



Fig. 45: The compression achieved by zeroing out a square of four pixels for each 8x8 block: 67840 elements out of 272640 are zeroed out (about 25%)



Fig. 46: The compression achieved by zeroing out a rectangle of 4x6 pixels for each 8x8 block: 101760 elements out of 272640 are zeroed out (about 37%)

Since the high frequency elements which are zeroed out are always incremented by one both along the rows and the columns, the amount of coefficients increases quadratically, as can be observed in Figure 47.

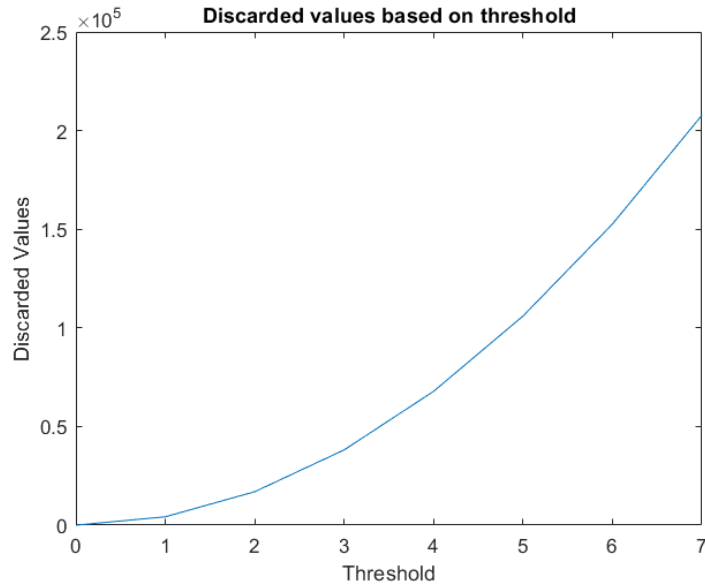


Fig. 47: The coefficients canceled, incrementing each time the number of rows and columns taken into consideration.

And the sizes of the image after increasing the values zeroed out:

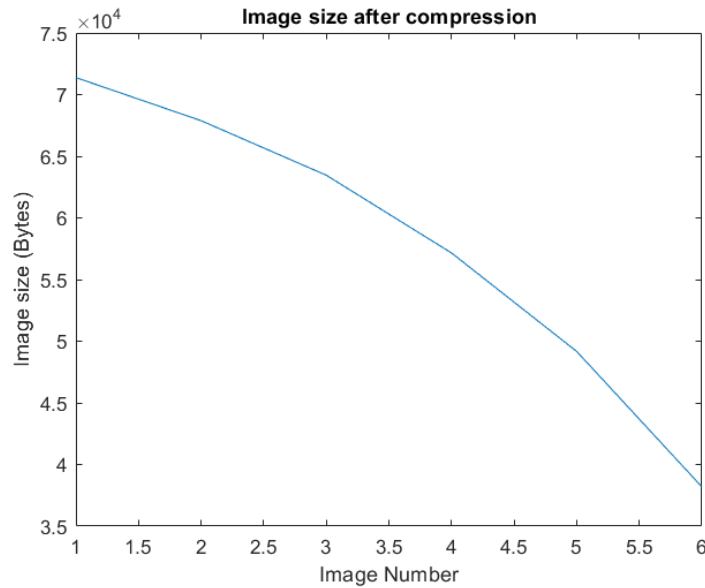


Fig. 48: The decreasing size of the image when compressed with JPEG, with the image quality being reduced.

4.3 Compression through Wavelet Transform

The Wavelet Transform can also be used to achieve image compression. The methodology used is not so different from the previous one (4.2.4), as the idea is to zero out some coefficient elements smaller than a specific threshold, and then, all the pixels could be represented through an Huffman Code, saving space.

Let's see how the Wavelet Transform works.

4.3.1 Wavelet Transform

The Fourier Transform is a useful tool in image processing, but it is somewhat limited, as it is not capable of keeping track of non-stationary signals (signals that are changing in time). Therefore, a new multiresolutional method is now introduced, which is the Wavelet Transform, which is able to describe both long-term and short term behaviour of a function. Furthermore, the Fourier Transform uses sinusoids as base functions, while the Wavelet Transform uses wavelets.

In general Wavelets use different *window* sizes, which try to capture the various frequency parts, corresponding to 'resolutions': that is why this technique is commonly referred to as multiresolutional.

But what is meant by window? A window is a decaying function, more precisely, a function that is 0 outside of a specific range.

All the different Wavelets are obtained by shifting and dilatating a *mother Wavelet* (ψ). A function $f(t)$ is described as

$$W(s, \tau) = \int_{-\infty}^{\infty} f(t) \frac{1}{\sqrt{s}} \psi\left(\frac{t - \tau}{s}\right) dt. \quad (51)$$

With the increase of s (stretch factor) the focus is applied to low frequency components, because the stretch of the wavelet behave as a long-term picture of the signal, while the high frequency components are captured by a shrinking of s , as a contraction of the wavelets result in poor long term behaviour recognition, in favour of an attention on the details. Moreover, a salient aspect is that all the different waves must have the same energy as the Mother Wavelet, and this is achieved with the scale factor $\frac{1}{\sqrt{s}}$, as proved in [14].

On the other hand, τ determines the position of the wavelet in the signal, so as to concentrate on all the specific parts of the signal.

Eq. (51) will result in a set of coefficients, indicating the correlation of the wavelet with individual sections of the signal: the higher the coefficient, the more the similarity. The Wavelet Transform can be Continuous, as well as Discrete. In case of image processing the discrete version is used, and it is proved that choosing scales and shifting as power of 2 will result in a rich wavelet decomposition anyways.

As described in [11], the strength of this method, is that it can provide a multi-level decomposition of the signal. In fact, the signal can be splitted into two components, which corresponds to the low-frequency and high-frequency parts of the signal, as can be seen in Figure 49.

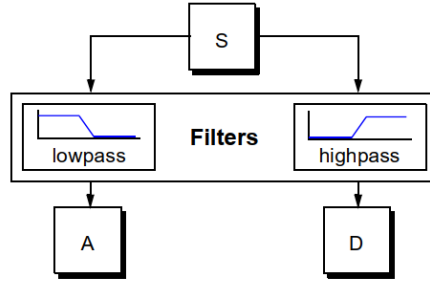


Fig. 49: Filtering of signal by dividing low-frequency and high-frequency components. (Source: [11])

The problem arises in terms of the number of samples given by the division in the different frequency components: for example, if the original signal samples are 1000, then we will end up having 1000 samples for the low-frequency part, and 1000 samples for the high-frequency part. Thereby, it is necessary to perform a downsampling, so as to have half the samples for low-frequency, and also half the samples for high-frequency. In such wise, aliasing (3.5.1) is introduced, but can be counteracted through an appropriate filter, or by adding zeros samples , as shown in Figure 50.

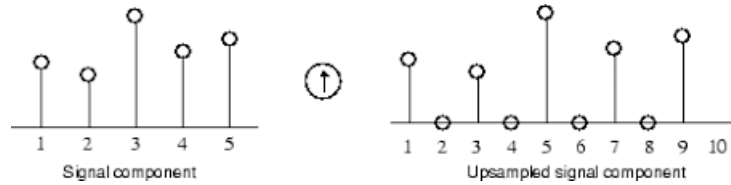


Fig. 50: Upsampling process by inserting zeros between samples [20].

If we apply a further division of the divided signal, a multi-level picture of the signal can be obtained, and it can be represented through a tree (Figure 51).

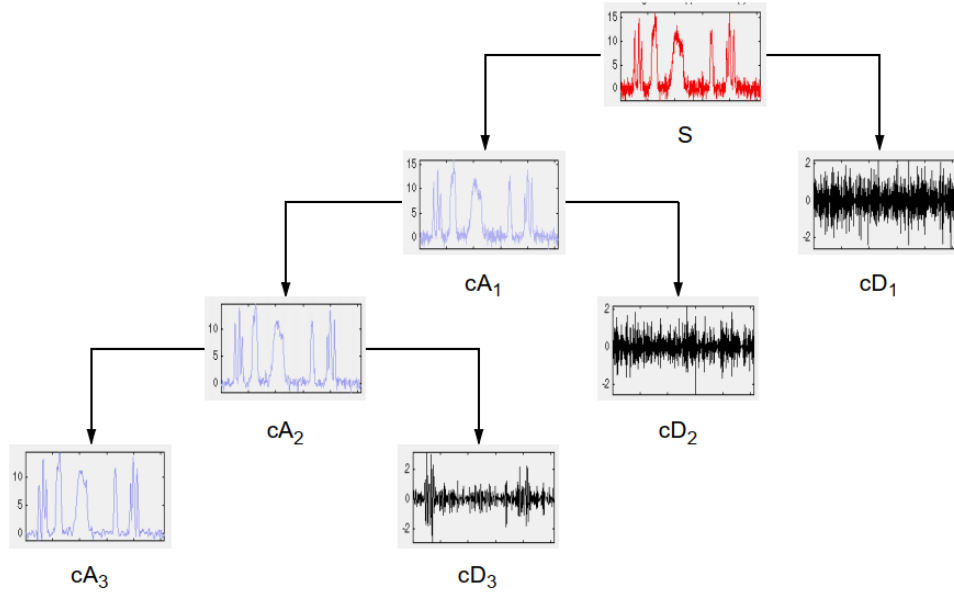


Fig. 51: Wavelet signal decomposed multiple times, providing a multiresolutional view. (Source: [11])

To reconstruct the signal, it is necessary to cope with the aliasing reached by the downsampling in the decomposition phase. This is achieved through upsampling and the application of a particular filter for each node in the wavelet signal decomposition. Then, each reconstructed leaf will have to be summed up. It is necessary to decompose the original image signal in multiple approximation and details, both on the rows and on the columns, as shown in Figure 52.

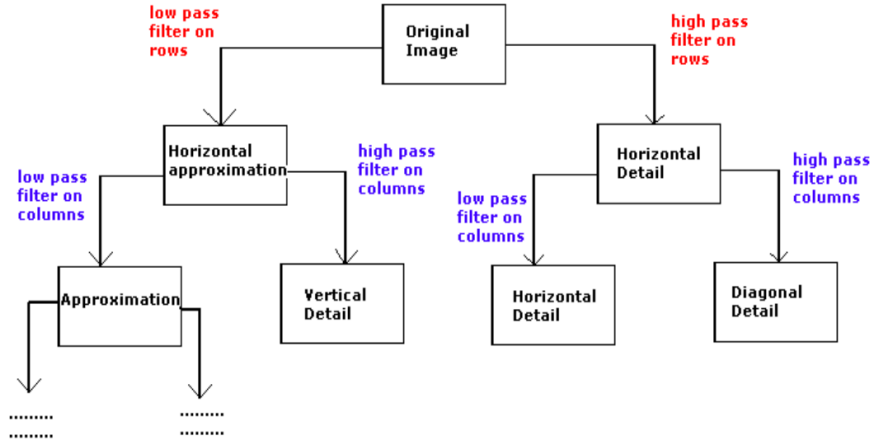


Fig. 52: Multiple decomposition of image through Wavelets (Source: [12])

Using the Wavelet Tool in MATLAB, it is possible to view the image signal decomposed in different levels (in this case 2 levels), with horizontal, diagonal, and vertical details, and their respective approximations (Figure 53).

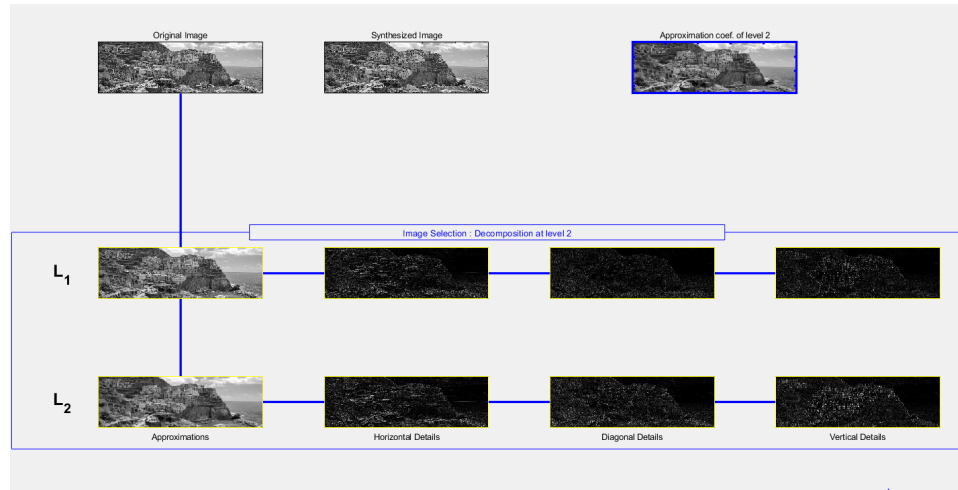


Fig. 53: In the above part the reconstructed images from the decomposition; in the section below Level 1 and 2 with their different details.

The wavelet decomposition can be achieved also through the following code:

Listing 11: Wavelet decomposition code

```

1 clear;close all;
2 %wavelet decomposition
3 img = rgb2gray(imread('example.jpg'));
4 image(img);
5 [C,S] = wavedec2(img, 2, 'bior3.7');
6
7 cA2 = appcoef2(C,S,'bior3.7',2);
8 cH2 = detcoef2('h', C, S, 2);
9 cD2 = detcoef2('d', C, S, 2);
10 cV2 = detcoef2('v', C, S, 2);
11 cA1 = appcoef2(C, S, 'bior3.7', 1);
12 cH1 = detcoef2('h', C, S, 1);
13 cD1 = detcoef2('d', C, S, 1);
14 cV1 = detcoef2('v', C, S, 1);
15
16 colormap(gray);
17 subplot(4,2,1); image(cA2);
18 subplot(4,2,2); image(cH2);
19 subplot(4,2,3); image(cD2);
20 subplot(4,2,4); image(cV2);
21 subplot(4,2,5); image(cA1);
22 subplot(4,2,6); image(cH1);
23 subplot(4,2,7); image(cD1);
24 subplot(4,2,8); image(cV1);

```

In script 11 the image is decomposed into two levels, each of the horizontal, diagonal and vertical details having their own coefficients. Now I will try to zero out the element smaller than a specific coefficients to see the result. The threshold is set to 50, and the difference between the coefficients is visible, but not so radical, in image 55 you can notice that some details vanished.

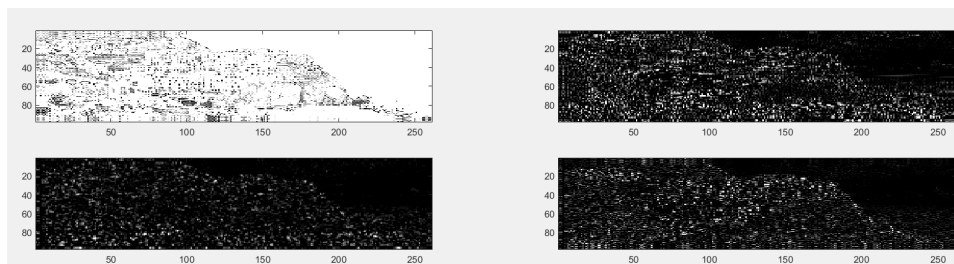


Fig. 54: Original details

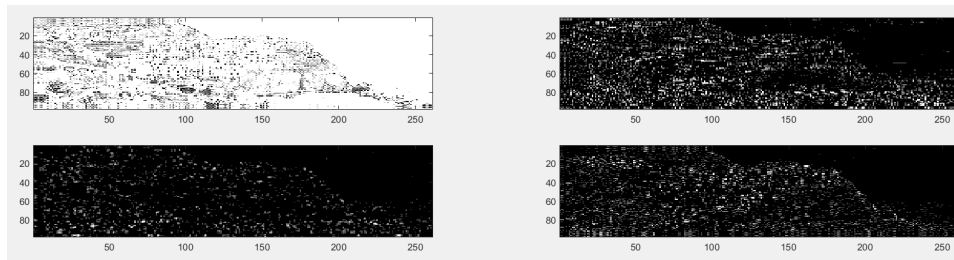


Fig. 55: Details after having zeroed out some coefficients

I used the code found in [18] to compress the image with wavelets.

Listing 12: Wavelet compression code

```

1  %compress the image with threshold from 1 to 100
2  thr = 1;
3  n = 5;
4  w = 'sym8';
5  [c,l] = wavedec2(img,n,w);
6  opt = 'gbl';
7  sorh = 'h';
8  keepapp = 1;
9  while thr < 100
10     [xd,cxd,lxd,perf0,perfl2] = wdencmp(opt,c,l,w,n,thr,sorh,
        keepapp);
11     figure
12     image(xd); colormap(gray);
13     title(strcat('Compressed Image — Global Threshold = ',
        num2str(thr)));
14
15     %store data to be plotted after the while loop
16     %compression ratio
17     plotData(thr) = perf0;
18
19     %store the compressed images
20     imageName = strcat('C:\Users\patri\Documents\UNI\year_3\
        image_compression\Scripts\waveletCompression\', num2str(
        thr), '.jpg');
21     imwrite(xd, imageName);
22     thr = thr + 1;
23 end
24 figure;
25 plot(1:99, plotData);

```

Now i will show the original image (56), and then the ones with threshold 25

(57), 50 (57), 75 (59) and 99 (60). It is noticeable that at first the quality does not decrease dramatically, but in the last two pictures the quality difference can be seen.



Fig. 56: Original image.



Fig. 57: Image compressed by zeroing out all the values less than 25.



Fig. 58: Image compressed by zeroing out all the values less than 50.



Fig. 59: Image compressed by zeroing out all the values less than 75.



Fig. 60: Image compressed by zeroing out all the values less than 99: the quality now is reduced a lot.

The compression ratio is similar as in [41](#), but in this case the curve is more roundish:

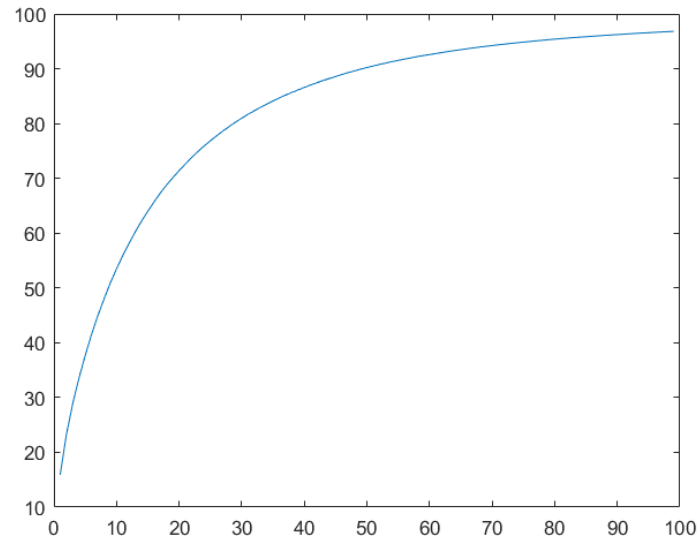


Fig. 61: Compression rate by increasing the threshold at each iteration: the curve is logarithmic.

And the size of the image follows this curve:

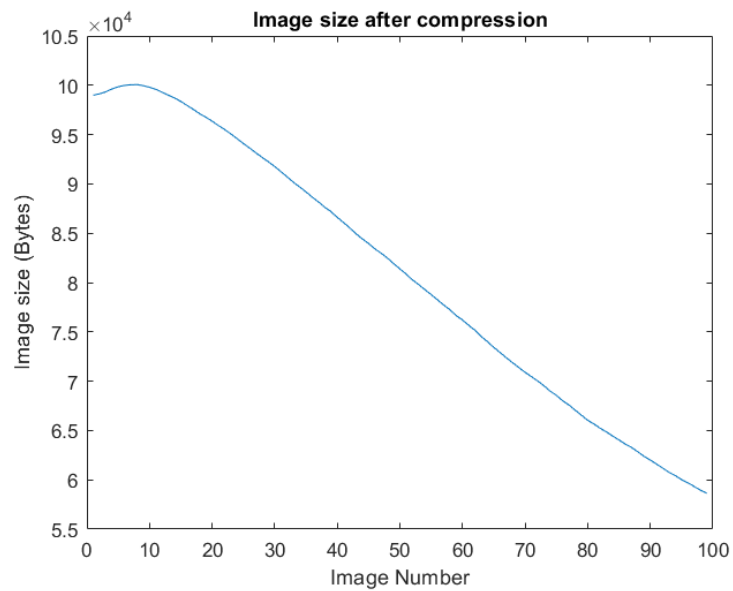


Fig. 62: Behaviour of the size of the image with different threshold values.

4.4 Comparison and Complexity of Compression Algorithms

We got through some various image compression techniques, thereby it would be noteworthy to compare them.

What must be pointed out is that the choice of a specific technique is based predominantly on the properties (like entropy, correlation between pixels, correlation between colors...) of the image which is taken into consideration.

4.4.1 Huffman Coding

Huffman Coding is a widespread compression lossless technique, and is utilized also in the lossy compression methods to encode data.

The complexity required to build an Huffman tree is $\mathcal{O}(N \log_2(N))$. In fact, for each insertion it can be proved that $\log_2(k)$ steps need to be performed, where k needs to be incremented every time a new level is added: the last added leaf level will have $k = \log_2(N)$ nodes to be travelled through to be added, while the previous ones will have $k = \log_2(N - 1), k = \log_2(N - 2), \dots, k = \log_2(2)$. So, we can rewrite the steps as $\log(k!)$. Looking at the *Stirling's approximation* (C), $\log_2(k!)$ can be reformulated as $\mathcal{O}(N \log_2(N))$. Then the access to the symbols can be achieved in logarithmic operations' complexity, as a tree traversal should be executed, where in the worst case the symbol to be retrieved is a leaf, while in the best case it is the root. Leastwise, if all the symbols need to be accessed, the most efficient algorithm to retrieve all of them would employ $\mathcal{O}(2^{\text{Height}} - 1)$ operations, which correspond to the number of nodes in a binary tree.

4.4.2 JPEG

JPEG compression should be applied on images without high contrast, because when the image is transformed into its frequency domain through the Discrete Cosine Transform, a lot of the high frequencies are cut, and those correspond to high contrast elements. The JPEG method is used widely, and consists of computing the Discrete Cosine Transform, which requires $\mathcal{O}(N^2 \log_2 N)$ steps (as found in [13]), and then, as data is encoded through Huffman Coding, and this, as described in 4.4.1, takes $\mathcal{O}(N \log_2(N))$ operations, so we have in total $\mathcal{O}(N^2 \log_2 N) + \mathcal{O}(N \log_2(N))$.

4.4.3 Wavelets

Wavelets are really functional when the image contains noisy and transient data, as wavelets are sensitive to change in time/space, and this results in being able of storing this kind of data in less storage space. On the other hand, smooth signals are not well compressed through wavelets. In this case it would be suggestable to manipulate them through the DCT or the Fourier

Transform.

To perform a Discrete Wavelet Transform the time complexity is $\mathcal{O}(N)$.

4.4.4 SVD

The SVD compresses by reducing the rank of a matrix: the time complexity is $\mathcal{O}(m \cdot n \cdot \min(m, n))$, and is strictly connected to the size of the matrix taken into consideration.

4.5 Operations comparisons

Compression methods compared		
Compression technique	Type	Algorithmic Costs
Huffman Coding	Lossless	$\mathcal{O}(N \log_2(N))$ (build tree) + $\mathcal{O}(2^{Height} - 1)$ (access tree)
JPEG	Lossy	$\mathcal{O}(N^2 \log_2 N) + \mathcal{O}(N \log_2(N))$
Wavelets	Lossy	$\mathcal{O}(m \cdot n \cdot \min(m, n))$
SVD	Lossy	$\mathcal{O}(m \cdot n \cdot \min(m, n))$

4.6 Memory storage saving

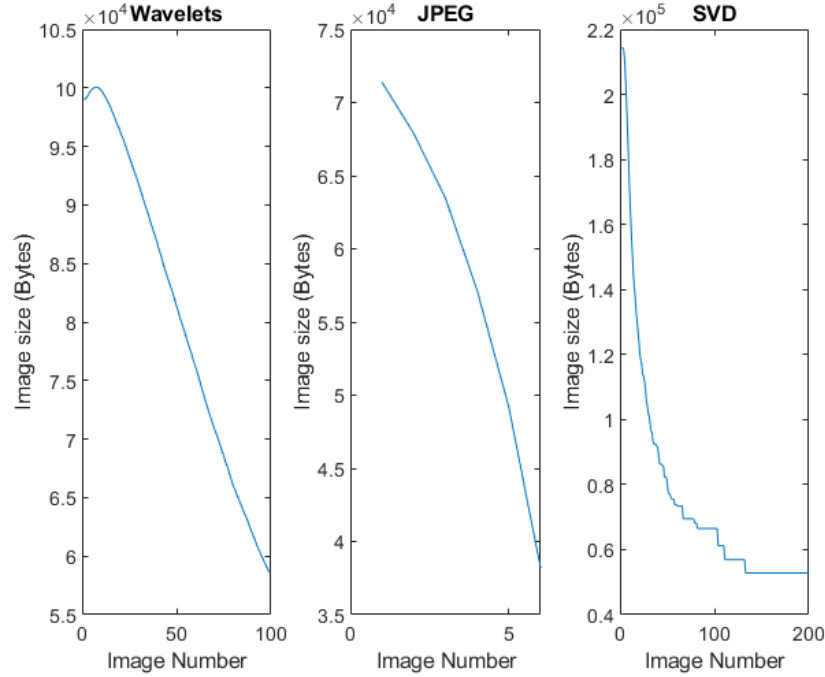


Fig. 63: How the size of the image file changes by quality degradation.

It can be noticed that the SVD curve decreases more rapidly than the other ones, also because the amount of dropped value increases more steeply, but I think the various lossy compression techniques are more or less comparable.

5 Conclusion

During this project I was able to learn to get confident with the MATLAB and LaTeX environments, and I will take advantage of this also in the future, because they are widely utilized in the Computer Science, and in the Sciences fields in general.

Furthermore, I have broadened my knowledge in some mathematical topics and in image processing field, which was my foreseen goal. The theory was followed by practical codes and examples, which I really enjoyed to prepare, and that benefitted the interiorization of the concepts, as well as providing a concrete application of what was learned.

In conclusion, I can admit that this was for me a worthwhile and constructive experience, also because the topics covered were of my pleasure, and

therefore, much appreciated.

A A small Background in Trigonometry

Lemma 4 $(\sin a \sin b)$.

$$\sin(a) \sin(b) = \frac{1}{2} \cos(a - b) - \frac{1}{2} \cos(a + b). \quad (52)$$

Proof. This can be proved by using the identities for $\cos(a - b)$ and $\cos(a + b)$

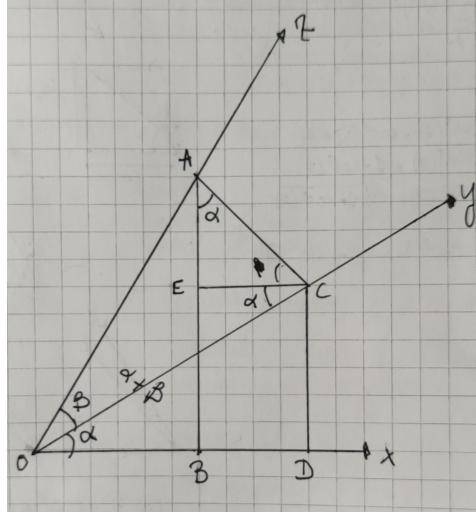


Fig. 64: A geometrical description of the proof

In the above image (Figure 64) it is given a geometric explanation for the identity $\cos(a+b) = \cos(a)\cos(b) - \sin(a)\sin(b)$

In fact, we can work with the sides of the various triangles and obtain the desired result

$$\begin{aligned} \cos(\alpha + \beta) &= \frac{OB}{OA} = \frac{OD - BD}{OA} = \frac{OD - EC}{OA} = \frac{OD}{OA} - \frac{EC}{OA} \\ &= \frac{OD}{OC} \frac{OC}{OA} - \frac{EC}{AC} \frac{AC}{OA}. \end{aligned} \quad (53)$$

As $\frac{OD}{OC}$ is the ratio between the cathetus adjacent to angle α and the hypotenuse OC , it can be substituted with $\cos(\alpha)$, $\frac{OC}{OA}$, on the other hand, is the ratio between the cathetus adjacent to angle β and the hypotenuse OA , and can be rewritten as $\cos(\beta)$. Hence, the term

$\frac{OD}{OC} \frac{OC}{OA}$ is $\cos(\alpha) \cos(\beta)$; Moreover, the term $\frac{EC}{AC}$ is the ratio between the cathetus non-adjacent to angle α and the hypotenuse AC, hence $\sin(\alpha)$, and the term $\frac{AC}{OA}$ is the ratio between the cathetus non-adjacent to angle β and the hypotenuse OA, hence $\sin(\beta)$

Plugging the new terms in $\cos(\alpha + \beta) = \frac{OD}{OC} \frac{OC}{OA} - \frac{EC}{AC} \frac{AC}{OA}$ we get

$$\cos(\alpha + \beta) = \cos(\alpha) \cos(\beta) - \sin(\alpha) \sin(\beta). \quad (54)$$

To find $\cos(\alpha - \beta)$ we can just consider $\cos(\alpha + (-\beta))$ and reverse all the signs of β in the equation above

$$\cos(\alpha + (-\beta)) = \cos(\alpha) \cos(-\beta) - \sin(\alpha) \sin(-\beta).$$

As cosine is an even function ($\cos(x) = \cos(-x)$) and sine is an odd function ($\sin(x) = -\sin(-x)$), we get

$$\cos(\alpha - \beta) = \cos(\alpha) \cos(\beta) + \sin(\alpha) \sin(\beta);$$

recalling Eq. (4) we can do a substitution of the terms on the right side

$$\begin{aligned} \sin(a) \sin(b) &= \frac{1}{2} \cos(a - b) - \frac{1}{2} \cos(a + b) && \leftrightarrow \\ \sin(a) \sin(b) &= \frac{1}{2} (\cos(a) \cos(b) + \sin(a) \sin(b)) - \frac{1}{2} (\cos(a) \cos(b) - \sin(a) \sin(b)) && \leftrightarrow \\ \sin(a) \sin(b) &= \frac{1}{2} (\cos(a) \cos(b) + \sin(a) \sin(b)) + \frac{1}{2} (-\cos(a) \cos(b) + \sin(a) \sin(b)) && \leftrightarrow \\ \sin(a) \sin(b) &= \frac{1}{2} (2 \sin(a) \sin(b)) && \leftrightarrow \\ \sin(a) \sin(b) &= \sin(a) \sin(b). \end{aligned}$$

□

Lemma 5 ($\int \cos^2(ax)$).

$$\int \cos^2(ax) = \frac{x}{2} + \frac{1}{4a} \sin(2ax) + c. \quad (55)$$

Proof.

$$\begin{aligned}\sin^2(ax) &= 1 - \cos^2(ax) \\ \int \sin^2(ax) &= \int 1 - \cos^2(ax) dx \\ &= \int 1 dx - \int \cos^2(ax) dx. \\ \text{As } \cos^2(ax) &= \frac{1 + \cos(2ax)}{2},\end{aligned}$$

we can proceed by integrating by parts :

$$\begin{aligned}\int \sin^2 dx &= x - \left(\frac{x}{2} + \int \frac{\cos(2ax)}{2} dx \right) \\ u = 2ax &\rightarrow \\ \frac{du}{2a} &= dx \\ \int \sin^2 dx &= x - \left(\frac{x}{2} + \frac{1}{2} \int \cos(u) \frac{du}{2a} \right) = \\ &= x - \left(\frac{x}{2} + \frac{1}{4a} \int \cos(u) du \right) = \\ &= x - \left(\frac{x}{2} + \frac{1}{4a} \sin(2ax) \right) = \\ &= x - \frac{x}{2} - \frac{1}{4a} \sin(2ax) = \\ &= \frac{x}{2} - \frac{1}{4a} \sin(2ax).\end{aligned}$$

□

B Unitary Transforms

Considering an $N \times N$ image, it can be represented as a linear combination of N^2 basis images. Take a vector x , containing N^2 elements. Then, its Unitary Transform y is given by

$$y = Ax \tag{56}$$

Where A is a matrix of size $N^2 \times N^2$. The same equation can be reformulated as

$$y(k, l) = \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} x[m, n] A(k, m; l, n), \quad 0 \leq k, l \leq N-1. \tag{57}$$

If A is orthogonal (hence, its transpose corresponds to its inverse, $AA^t = I$, with real values), then we use the following inverse operation to find $x[m, n]$

$$x(m, n) = \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} y[k, l] A(k, m; l, n), \quad 0 \leq m, n \leq N-1. \quad (58)$$

C Stirling Approximation

As explained in [17], $\ln n!$ can be approximated in this way:

$$\begin{aligned} \ln n! &= \ln 1 + \ln 2 + \dots + \ln n = \sum_{k=1}^n \ln k = \\ &\approx \int_1^n \ln(x) dx = [x \ln x - x]_1^n = n \ln n - n + 1 \\ &\approx n \ln n - n. \end{aligned}$$

References

- [1] Singular Value Decomposition by *Professor Bruno Carpentieri*
Course: *Computational Mathematics*
Academic Year: *2018-2019*
- [2] <https://www.image-engineering.de/library/technotes/745-how-does-the-jpeg-compression-work>
- [3] Human Vision and Perception by *Mahalakshmi Ramamurthy and Vasudevan Lakshminarayanan*
- [4] Practical Image and Video Processing Using MATLAB®. By *Oge Marques*
- [5] Digital Image Processing by *Rafael C. Gonzalez and Richard E. Woods*
- [6] The Fourier Transform and its Applications by *Prof. Brad Osgood*
Electrical Engineering Department Stanford University
- [7] Wikipedia
- [8] <https://www.nti-audio.com/en/support/know-how/fast-fourier-transform-fft>
- [9] <https://lpsa.swarthmore.edu/BackGround/ImpulseFunc/ImpFunc.html>
- [10] CS425 Lab: Frequency Domain Processing

- [11] Wavelet Toolbox For Use with MATLAB® by *Michel Misiti Yves Misiti Georges Oppenheim Jean-Michel Poggi*
- [12] *Image Compression Using Wavelets* by *Karen Lees May 2002*
- [13] Quantum Discrete Cosine Transform for Image Compression by *Chao-Yang Pang¹, Zheng-Wei Zhou, and Guang-Can Guo*
- [14] Still Image and Video Compression with Matlab by *Thyagarajan*
- [15] <http://cs.boisestate.edu/~alark/cs464/lectures/AntiAliasing.pdf>
- [16] <https://people.eng.unimelb.edu.au/ammoffat/abstracts/compsurv19moffat.pdf>
- [17] <https://mathworld.wolfram.com/StirlingsApproximation.html>
- [18] <https://it.mathworks.com/help/wavelet/ug/data-compression-using-2d-wavelet-analysis.html>
- [19] <https://it.wikipedia.org/wiki/YCbCr>
- [20] <https://it.mathworks.com/help/wavelet/gs/wavelet-reconstruction.html>